A Comprehensive Survey of World Models for Coding

Keon Kim[1]

[1]Independent Researcher

Code & Survey arXiv

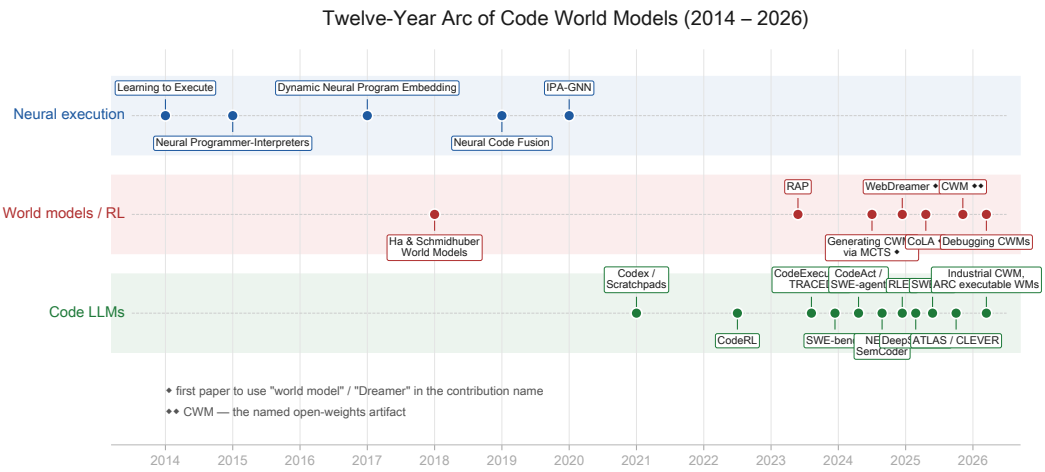

Figure 1: Twelve-year arc of code world models. Three parallel lineages — neural execution, world models / RL, and code LLMs — converge in 2025–2026 around the named artifact CWM. ✦ marks the first paper to use "world model" / "Dreamer" in a contribution name; ✦✦ marks CWM as the named open-weights artifact.

0.1 Abstract

A *world model* is an internal predictor over environment dynamics, used to imagine the consequences of actions. In coding, the environment is the program: its runtime state, execution trace, filesystem, tests, and developer task. Twelve years after Zaremba & Sutskever asked whether a network could execute code [1], and seven months after Meta FAIR released CWM [2] as the first open-weights LLM branded a Code World Model, the question has shifted. Internal models of execution are demonstrably *learnable*; whether they are *necessary*, *architecturally separable*, or *causally responsible* for coding-agent gains is unsettled.

This survey synthesizes 184 papers under two definitions — a permissive *descriptive* one matching field usage, and a strict *normative* one requiring forward-prediction machinery. It traces a twelve-year arc; builds a three-axis taxonomy (functionality × temporal × representation); produces system cards for thirteen representative systems; assembles protocol-stratified tables for SWE-bench, CRUXEval, web agents, and formal verification; develops seven critical theses; and lists open problems. The defensible empirical claim is that execution-grounded supervision improves code agents on runtime-reasoning benchmarks. The defensible critical claim is that broader inferences to "world models for coding" remain underdetermined.

Preprint.

0.2 Theses Summary

The seven critical theses developed in §17, each grounded in a specific disconfirmation from the corpus:

1. The "world model" label has become overloaded. Under the field's permissive descriptive definition (D, §3.1), CWM, TRACED, SemCoder, LLM-JEPA, and DyMo are all "world models" despite being architecturally standard LLMs with enriched training. A strict normative definition (N1/N2/N3, §3.1) admits only systems with explicit forward-prediction machinery: neural dynamics models (N1, no code exemplar), latent-action rollout models such as CoLA (N2), and synthesized executable simulators such as GIF-MCTS and ARC executable WMs (N3).
2. Trace pretraining has a causal-isolation problem. "Do Code Semantics Help?" [3] finds no trace representation consistently improves code synthesis across multiple backbones; CWM's 65.8% SWE-bench Verified is confounded between trace mid-training, ForagerAgent trajectories, and RL.
3. The Dreamer-for-code gap may be a non-problem. Vision needed latent rollouts because pixels are expensive; Python state is small and CPython is free, so the architectural pressure does not transfer. CWM in token space reaches state-of-the-art at 32B.
4. PRMs are critics, not world models. A learned evaluator of partial trajectories cannot roll out future states; the conceptual conflation dilutes the world-model vocabulary.
5. "General Agents Contain World Models" [4] is weaker than its title. The theorem applies under restrictive assumptions (full observability, stationarity, deep-composite-goal regret bounds) that SWE agents demonstrably violate.
6. The verifier-grounded lineage is the actual leading edge, but scoped. ATLAS, Re:Form, CLEVER, VeriStruct, AutoRocq produce machine-checkable correctness. Scoped: they lead on synthesis-from-spec (e.g., ATLAS DafnySynthesis 65.8% pass@5); they do *not* lead on end-to-end verified codegen from NL (CLEVER ?1/161 Lean).
7. The evaluation gap is the structural reason the field looks confused. No benchmark holds policy fixed and varies WM quality. Models with 85–98% output-prediction accuracy still produce traces with systematic errors throughout — outcome accuracy decouples from process fidelity.

0.3 1. Introduction

Autoregressive code LLMs generate tokens conditioned on syntactic context. Correct programs live in two worlds: a *syntactic* world of tokens and a *semantic* world of values, control flow, side effects, and developer intent. The world-model framing — imported from model-based reinforcement learning, where it names an internal predictor of environment dynamics used to imagine action outcomes — is the field's bet that the gap between these two worlds closes when the network has been trained on what code does rather than only on what code looks like.

Two adjacent surveys cover non-overlapping ground. A Survey on LLMs for Code Generation [5] maps the code-LLM space without the world-model lens. Understanding World or Predicting Future [6] maps world models in general without the code lens. A Comprehensive Survey on World Models for Embodied AI [7] maps embodied world models but excludes the coding domain. The intersection — the subject of this document — has cohered only recently into a recognizable program.

0.4 2. Methodology

Corpus. 184 PDFs in `papers.json`, assembled in four iterative passes between March and May 2026. Seed: CWM [2]. Each pass expanded along a different axis — citation BFS via Semantic Scholar, targeted topic search for thin subdomains (latent-action WMs, safety, symbolic

verification, agent memory, REPL-grounded), a 2026-specific sweep, and a final pass on reasoning models, PRMs, decompilation, diffusion code, mech-interp, ARC, and hardware/RTL. Of ~250 candidates considered across passes, 184 were accepted.

Inclusion. A paper enters the corpus if it credibly intersects *both* world-model/state-tracking architectures and code generation, debugging, repair, or agentic coding. Date cutoff: arxiv submissions through 2026-05-15. Pure code-LLM papers without a world-model angle and pure vision world models (DreamerV1–V3, V-JEPA, Genie) are excluded except where cited as precedent.

Limitations. No inter-rater reliability — one curator did the taxonomy coding. Inline numerical claims in §§6–14 rely on author abstracts and prior reading; §16 numbers were verified against source PDFs. Recency bias: 60% of the corpus is 2025 or later. Anglocentric arxiv-first selection skips Chinese-language preprint servers and industry technical reports. A fifth pass would change the count by ±15 without materially changing conclusions.

0.5 3. Defining a World Model for Coding

The literature uses "world model" in two distinct senses that this survey will carefully separate.

0.5.1 3.1 Two definitions

Definition D (descriptive, permissive). A *world model for coding* is any code LLM whose training objective concretely encodes program semantics — execution traces, runtime state, environment feedback, or simulated outcomes — in addition to or instead of source-token prediction. This is the field's current usage. Under D, CWM, TRACED, SemCoder, LLM-JEPA, DyMo, RLEF, and most papers in the corpus are world models.

Definition N (normative, strict). A *world model for coding* is a system with an explicit, separable forward-prediction mechanism. We split N into three sub-types:

- N1 — Neural dynamics model. A learned `W : (state, action) → next_state` instantiated as a distinct architectural component with latent recurrent state, separate dynamics head, or inverse model. Dreamer-class. No code exemplar in the corpus.
- N2 — Latent-action model. A learned action abstraction over a base LLM, with the LLM serving as transition model in compressed action space, supporting rollout or tree search over latent actions. CoLA [8] is the canonical example.
- N3 — Synthesized executable simulator. The world model is an executable program (Python, DSL) synthesized by an LLM and run against ground-truth transitions during planning. GIF-MCTS [9], WorldCoder, Executable WMs for ARC-AGI-3 [10].

Each N-subtype has a distinct architectural commitment. N1 commits to learned latent dynamics; N2 commits to a discrete latent-action space; N3 commits to program synthesis as the dynamics. Conflating them — as the loose "Dreamer-for-LLMs gesture" framing did — obscures which architectural bet is being made. Under any N-subtype, CWM, TRACED, SemCoder, and most of the corpus do *not* qualify.

The two definitions agree on the empirical fact that execution-grounded supervision helps. They disagree on whether that grounding is correctly named a *world model* in the model-based-RL sense.

Aspect	Definition D (descriptive)	Definition N (normative)
Granted to	Any LLM trained with execution-related signal	Only systems with explicit forward-prediction module
Includes CWM	Yes	No (architecturally a Llama-class decoder)
Includes TRACED, SemCoder, NExT	Yes	No (auxiliary heads on a Transformer)

Aspect	Definition D (descriptive)	Definition N (normative)
Includes CoLA, GIF-MCTS	Yes	Yes
Includes PRMs (ExecVerify, ThinkPRM)	Often grouped	No (critics, not forward predictors)
Includes verifiers (Lean, Dafny, Z3)	Sometimes grouped	No (deterministic oracles, not learned WMs)
Includes Dreamer / V-JEPA (vision precedents)	Yes	Yes

This survey uses D throughout the catalog sections (§§6–16) because that is how the literature talks, and N in §17 because the critical synthesis depends on it. We mark the switch each time it matters. The split has been called for in prior reviews of the field and the two-definition framework is, in our reading, the cleanest resolution.

0.5.2 3.2 What is modeled

Orthogonal to D-vs-N, a fourth question asks *what* is modeled. The corpus splits across:

- variable values and stack frames (CWM, CodeExecutor)
- linear or branching execution traces (NExT, SemCoder, TRACED)
- test outcomes and runtime errors (LEVER, RLEF)
- environment/OS/web state (WebDreamer, Dyna-Think)
- repository state (RepoGraph, Understanding by Reconstruction)
- developer task or specification (ATLAS, Re:Form)
- adversarial behavior (Double Life of CWMs)

A given system typically commits strongly to one or two of these.

0.5.3 3.3 Behavioral vs architectural classification

CWM occupies an instructive position. *Behaviorally*, it emits stack frames and variable bindings, which feels like an "explicit symbolic world model." *Architecturally*, it is a 32B Llama-class decoder with no separate dynamics head, RSSM, or inverse model. The two readings are both correct: CWM is behaviorally explicit but architecturally implicit. Earlier framings in the literature (and earlier drafts of this survey) collapsed these into a single "explicit WM" label; we recommend separating them. SemCoder, NExT, and TRACED follow the same pattern.

0.6 4. Twelve Years of Code World Models

The lineage is best understood as a sequence of inheriting questions. Each era's answer dissolved the previous era's bottleneck and exposed the next.

(See Figure 1 at the start of the paper for the timeline diagram.)

Diamonds (⍰) mark moments where "world model" enters the *name* of the contribution.

Pre-2020 — Can a network execute code? Zaremba & Sutskever's Learning to Execute [1] handed an LSTM character-level Python and asked it to predict output. The model worked on straight-line programs with bounded loops, but only with a curated curriculum, and only because the LSTM's constant memory was just enough to simulate the interpreter when the interpreter ran in constant memory too. Everything that followed in this lineage tried to escape that curse. Neural Programmer-Interpreters [11] and the Differentiable Forth Interpreter built differentiable program counters and call stacks — the bet that the right architecture would close the gap. Dynamic Neural Program Embedding [12] made the inverse move: run the real interpreter, embed the resulting state traces. Neural Code Fusion [13] and IPA-GNN [14] extended the GNN-over-execution playbook to the point where attention played the role of the program counter. By 2020 the lineage had answered its question — yes, a neural network can play interpreter, but only when the interpreter is encoded into its architecture, and these architectures

did not transfer to Python, C, or assembly at corpus scale. Ha & Schmidhuber's World Models paper [15] had already named for vision and RL exactly the pattern this lineage was reaching for. The vocabulary existed; the coding community had not yet borrowed it.

2020–2022 — Can the network's training include execution? The era's reframing was simple: instead of *can the network execute*, ask *can we train a normally-shaped Transformer on enough execution evidence that semantics seep into its weights*. Codex [16] and MBPP [17] made code generation a real engineering target. Show Your Work / Scratchpads [18] made the decisive move: a Transformer that could not predict a program's output could predict it perfectly if it was allowed to emit the intermediate computation first. Same trick Dynamic Neural Program Embedding had pulled, now at LLM scale, in token space, without architectural surgery. The neural-as-interpreter program of 2014–2020 did not scale beyond toy languages; the next era's move was to bake execution into training data rather than architecture.

2023 — Trace pretraining as a named recipe. CodeExecutor [19] made the recipe explicit: mutate competitive-programming submissions, run them in a sandbox, capture per-line state tokens, train a transformer to emit the trace from source. TRACED [20] generalized this to a pretraining auxiliary that any code-LLM could absorb. CRUXEval [21] provided the canonical input/output-prediction benchmark and suddenly there was a number that captured "does this model understand what code does, as opposed to what code looks like." In parallel, Reflexion [22] and Self-Debug [23] showed that an LLM's mistakes could be fed back to itself as natural-language critiques, LEVER [24] used execution to verify candidate generations during decoding, and RAP [25] framed the LLM itself as a world model and ran MCTS over its imagined rollouts. The year's unifying insight: execution traces are an auxiliary objective, not a separate model.

2024 — From models that simulate to agents that act. SWE-bench [26] replaced "write a function that passes a unit test" with "fix a real GitHub issue in a real repository." CodeAct [27] claimed the agent's action space should be Python code itself. SWE-agent [28] shipped the harness. NExT [29] inlined traces into the agent loop. RLEF [30] fed execution outcomes back as RL rewards. WebDreamer [31] transplanted Dreamer-style imagination to digital agents. Generating Code World Models via MCTS [9] introduced the literal phrase "Code World Models." What unified the year was a structural shift in *where* the world model lives: in 2023 it lived in the weights, surfaced through trace prediction; in 2024 it lived in the loop, in the agent's behavior under environmental feedback.

2025 — The CWM moment. DeepSeek-R1 [32] opened the year by showing that pure reasoning-RL on verifiable rewards could reach the frontier on math and code. SWE-RL [33] applied the same recipe to full SWE traces. CoLA [8] made the first concrete attempt at a Dreamer-for-LLMs: inverse-dynamics over latent actions, then RL over a learned codebook. LLM-JEPA [34] ported LeCun's joint-embedding predictive objective to language. General Agents Contain World Models [4] supplied a theorem: any agent satisfying a regret bound on goal-conditioned tasks must have learned a predictive model of its environment. Then in October, Meta FAIR released CWM [2] — a 32B open-weights model mid-trained on 5T tokens of Python execution traces and ForagerAgent trajectories from Dockerized repositories. The thing the 2014 LSTM was trying to be was now a downloadable checkpoint.

2026 — Stress-testing and broadening. With the artifact in hand, the field pivoted to critique and generalization. Debugging Code World Models [35] catalogs CWM's failures on long traces and string-state representation. Demystifying Errors in LLM Reasoning Traces [36] audits where trace-trained models hallucinate. Industrial CWM [37] and Parallel-Code WMs [38] generalize the recipe to Verilog/GPU and parallelism semantics. Executable World Models for ARC-AGI-3 [10] brings the generative-environment flavor to abstract visual reasoning. Reinforcement World Model Learning for LLM Agents [39] flips the standard recipe by training the WM rather than the policy.

The arc. Across twelve years: *can a network execute code?* → *can a network's training include execution?* → *can an LLM agent simulate its environment?* → *is the world model a named artifact rather than a metaphor?* Each era's answer dissolved the previous bottleneck and exposed the next. Architecture gave way to data; data gave way to agency; agency gave way to artifacts. What was a half-philosophical question in 2014 — *does this network understand what code does* — became, in 2025, an operational one with an open-weights baseline. The field has not finished.

The 2026 critique wave shows the artifact is brittle in ways the 2014 LSTM was never asked to be. But the trajectory is now legible.

0.7 5. Taxonomy: Three Axes of Code World Models

Two cuts at the taxonomy are useful, and they are complementary. The first is a *lineage* cut — which research thread produced the system — and is the basis for §§6–13. The second, more durable cut adapts the three-axis framework of Li et al. ([7], *A Comprehensive Survey on World Models for Embodied AI*) to the code domain. The lineage map is below; the three axes are developed in §§5.1–5.3.

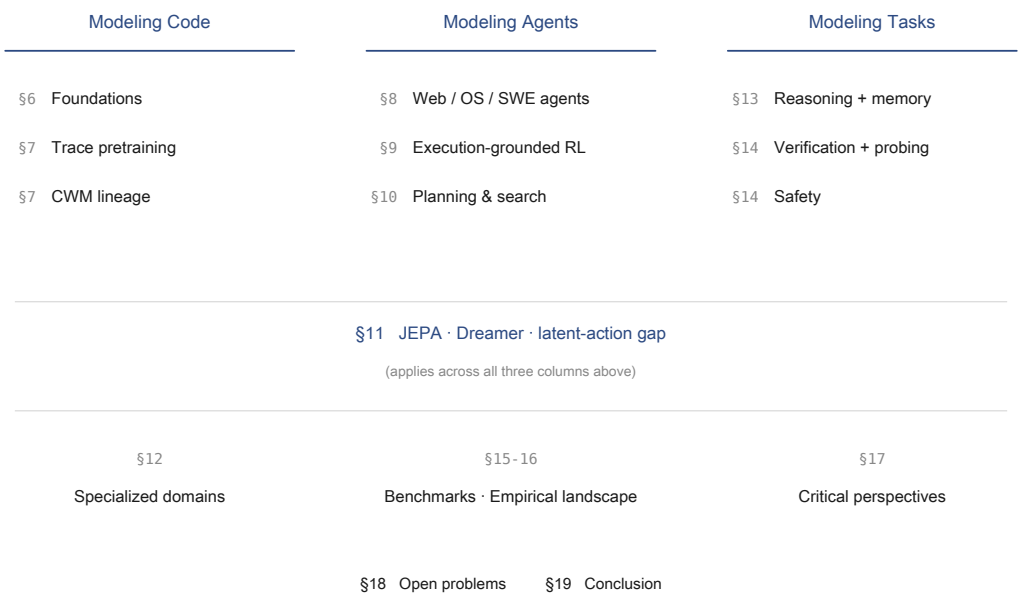

Figure 2: Taxonomy of world models for coding. Three modeling axes (code, agents, tasks) bridged by JEPA / Dreamer / latent-action discussion in §11. Specialized domains and synthesis chapters sit below.

Adjacent WM surveys converge on overlapping splits that the three-axis framework subsumes as projections. Ding et al. [6] split top-level by *implicit representation* vs *future prediction*. JiahuaDong's awesome-list organizes by *paradigm*: RL-based / observation-generative / latent-space / object-centric. knightnemo's list surfaces *pixel vs mesh vs latent* as cross-cutting tags. The three axes — functionality, temporal modeling, and representation — capture the choices a system makes regardless of which lineage it belongs to.

0.7.1 5.1 Axis 1 — Functionality

- Decision-coupled WMs model only the slice of the world relevant to acting on it. CWM [2], RLEF [30], and WebDreamer [31] are decision-coupled — their WMs exist to enable code generation, RL planning, or web navigation respectively. CWM does not predict global filesystem state; it predicts the next Python frame *because* the next action depends on it.
- General-purpose WMs model the environment without reference to a particular task. The "general agents contain world models" theorem [4] is the abstract limit. In the corpus, only the largest CWM-class models with broad mid-training approach generality; most "code world models" are decision-coupled to a sub-task (repair, completion, agent control).

0.7.2 5.2 Axis 2 — Temporal Modeling

- Sequential simulation/inference. Step-by-step autoregressive rollout. CWM, NExT, SemCoder, all the trace-pretraining systems, and most LLM-as-WM planners (RAP, WebDreamer in its MPC loop) live here. The state is updated one timestep at a time. The vision analog is RSSM (Hafner et al., DreamerV1–V3).
- Global difference prediction. Predict the entire future state at once, in parallel. The vision analog is video-diffusion or masked-JEPA. In code, this fits diffusion code models (DiffuCoder, [40]; Dream-Coder 7B, [41]) where the next state is sampled jointly rather than autoregressively, and the "specification is the program" framing [42] where the entire trace is the spec.
- Static, no-trace. Some systems (SemCoder's static mode, the trace-free baselines in "Do Code Semantics Help?") explicitly drop temporal modeling at inference, reducing to single-shot prediction.

0.7.3 5.3 Axis 3 — Representation

This is the axis where the WM literature has converged most strongly, and where the code-WM literature is most uneven. Adapting Li et al.'s four-category split (GLV / TFS / SLG / DRR) to code yields five classes, of which only two are well-populated.

Class	Encodes the world as	Vision analog	Code exemplars
Token Sequence (TS)	Discrete or continuous token streams with execution traces, variable bindings, or rationales interleaved with source	Token-as-pixel (IRIS, TWM, Genie, Sora)	CWM [2], CodeExecutor [19], TRACED [20], NExT [29], SemCoder [43] — the dominant code-WM mode
Global Latent Vector (GLV)	A compact vector updated recurrently, encoding the entire program/agent state	RSSM (Hafner et al., DreamerV1–V3)	No clean exemplar. CoLA [8] introduces a learned action codebook but is otherwise a standard LLM, not RSSM-style
Spatial / Structural Grid (SLG)	A geometric or structural grid (BEV/voxel in vision; AST, call-graph, CFG in code)	OccWorld, DriveWorld	No exemplar. RepoGraph [44] uses a static dependency graph but does not predict over it as a WM
Decomposed Object / Slot (DOR)	Distinct persistent latent slots for objects in the world	SlotFormer and object-centric WMs	No exemplar. No code-WM models variables, scopes, or classes as discrete persistent slots
Synthesized executable (N3)	The world model *is* an executable program, synthesized rather than learned	(orthogonal to vision)	GIF-MCTS [9], WorldCoder, Executable WMs for ARC-AGI-3 [10]

Verifiers (Lean, Dafny, Z3) and PRMs are intentionally absent from this table. A verifier is not a *representation* of the world; it is a *grounding oracle* over candidate artifacts. A PRM is a *backward-looking critic*, not a forward predictor. Both belong in §5.4 below.

Three white spaces. The Dreamer-style GLV, the object-centric DOR, and the spatial-grid SLG representations have not been instantiated for code, despite being mature for vision. §18 lists

these as open problems, with the caveat that not all three are equally promising — the Dreamer-style gap is contestable (§17.3), while the object-centric and structural-grid gaps look more genuine.

0.7.4 5.4 Grounding mode (orthogonal to representation)

A second classification asks how each system's predictions are validated. This is the analog of Li et al.'s "reality" column and addresses the decoupling between WM-head accuracy and downstream task success that DyMo [45] and §17.7 develop.

Grounding mode	Definition	Code exemplars
None / self-report	Predictions never validated against ground truth	RAP [25], basic LLM-as-WM planners
Execution-grounded	Predictions checked against real interpreter / runtime	CWM [2], TRACED [20], NExT [29], SemCoder [43], RLEF [30]
Verifier-grounded	Outputs checked by Lean / Dafny / Verus / Rocq / Z3	ATLAS [46], Re:Form [47], CLEVER [48], VeriStruct [49], AutoRocq [50]
Synthesized-simulator-checked	Synthesized world model checked against held-out transitions	GIF-MCTS [9], Executable WMs for ARC-AGI-3 [10]
Critic-grounded (not a WM)	Backward-looking value/quality score, no rollout	ExecVerify [51], SWE-PRM [52], ThinkPRM [53] — listed for contrast; these are critics, not WMs (§17.4)

Grounding mode is orthogonal to the representation axis: a token-sequence representation can be execution-grounded (CWM) or self-report (RAP); an N3 synthesized-simulator can be checked against transitions (GIF-MCTS) or never validated. The "WM fidelity" question of §17.7 is, structurally, a question about whether a system's grounding mode is strong enough to falsify a bad WM at training time.

0.8 6. Foundations: Neural Execution as Implicit World Modeling

Zaremba & Sutskever's Learning to Execute [1] established both feasibility and brittleness. Show Your Work — Scratchpads [18] is the hinge moment: by training a Transformer to emit intermediate computation states, the authors recovered much of the LSTM-era execution-prediction performance at scale, presaging the trace-pretraining lineage of §6. CRUXEval [21] and REval [54] provide the canonical execution-reasoning benchmarks. The lesson the field absorbed: *replacing* the interpreter with a neural network is harder than *augmenting* a transformer with interpreter-style supervision. Modern systems all take the latter path.

0.9 7. The Trace-Pretraining and CWM Lineage

0.9.1 7.1 Trace-pretraining as a recipe

CodeExecutor [19] trains a Transformer to simulate Python execution token-by-token. TRACED [20] adds dynamic-state supervision to a code-LLM pretraining mix. NExT [29] formats traces as natural-language rationales, letting a chat-style LLM reason about runtime behavior via chain-of-thought. SemCoder [43] generalizes to "monologue reasoning" linking source-text to execution state.

The 2025 wave consolidated and stress-tested the approach. "What I cannot execute, I do not understand" [55] trains and evaluates LLMs explicitly on traces with dynamic scratchpads, pushing Llama-3.1-8B from 37.8% to ~80% on CRUXEval-O. Code Execution as Grounded Supervision [56] repurposes line-by-line traces as verifiable CoT. Self-Execution Simulation [57] lets the model train on its own execution predictions. Demystifying Errors in LLM Reasoning Traces [36] audits where trace-trained LLMs fail. "Do Code Semantics Help?" [3] is the most damaging paper in the lineage: a comprehensive ablation across DeepSeek-Coder, LLaMA-3, and Gemma-2 with five trace representations finds that *no single representation consistently improves code generation*, and in 7 of 9 synthesis settings the no-trace baseline wins or ties.

0.9.2 7.2 TRACED [20]

TRACED augments a RoBERTa/UnixCoder pre-training mix with two execution-grounded heads on top of standard MLM: per-line program-state classification (variable type and quantized value over 30 bins) and per-line execution coverage. Trained on ~121k C traces from CodeNet collected via gdb, it shows that *quantized* variable-value prediction is a viable auxiliary signal — concrete values lose to discretized bins. On static execution estimation, full-path accuracy rises from UnixCoder's 63.7% to 71.6%, and downstream clone retrieval and defect detection improve modestly. The essence: trace prediction as a pre-training side objective, not a separate model.

0.9.3 7.3 NExT [29]

NExT inlines execution traces into source as Python-style comments (`# (k) varA=...; varB=...`) and trains PaLM 2-L on (rationale, fix) candidates via STaR-style self-training — sample 32 candidates per problem, accept those that pass unit tests, SFT on the accepted set, repeat. After ten iterations Mbpp-R pass@1 climbs from 23.2% to 49.3% (+26.1 absolute). The result that matters most for the rest of the survey: NExT *retains* a +17 absolute gain (23 → 40.8) on Mbpp-R when traces are *removed at inference*, which makes it the cleanest example of trace pretraining transferring to non-trace inference on a repair task.

0.9.4 7.4 SemCoder [43]

SemCoder formalizes "monologue reasoning" linking four code modalities — natural-language description, source, operational trace, and abstract input-invariant constraints — under a single NTP objective with rejection-sampled training data (the PYX corpus). Distinctive features: forward *and* backward monologues (NExT is forward-only), abstract-semantics constraints rather than concrete state at every step, and entirely static inference. SemCoder-1.3B reaches CRUXEval-I/O 63.6/65.1 vs GPT-3.5-turbo's 50.3/59.0, and a monologue-format ablation beats Scratchpad and NExT.

0.9.5 7.5 CWM [2]

CWM is a 32B dense decoder-only Transformer with grouped-query attention, sliding-window blocks, RoPE — architecturally Llama-class. What earns it the "world model" name is the mid-training datamix: 5T tokens of Python observation-action traces (120M traced functions, 262k CodeContests traces, 70k repo-level traced commits, 75M natural-language rewrites) plus 3M ForagerAgent SWE trajectories from 10.2k Dockerized repositories. Tokens are formatted so next-token prediction *is* next-state prediction at line granularity. CWM reaches 65.8% on SWE-bench Verified with test-time scaling (best@16 over 40 verifier-reranked samples), 94.3% on CRUXEval-Output. Its second technical contribution is *Activ*, which uses GitHub Actions CI to scale executable repository images. CWM is behaviorally explicit (emits stack frames) but architecturally implicit (no separate dynamics head).

Important caveat (developed further in §17): the 65.8% headline is *not* pure pass@1 but best-of-16 with verifier reranking. Pure pass@1 is approximately 53–55%. The trace-mid-training contribution is not causally isolated from the ForagerAgent-trajectory contribution and from the joint-RL contribution. Without an ablation removing one while holding the others fixed, the "world model" component's causal role is unfalsifiable.

0.9.6 7.6 Direct descendants of CWM

- Debugging Code World Models [35] — probes where CWM fails on long traces and string state; finds long-horizon failures are dominated by *action hallucination*, not state-propagation error.
- Learning Reasoning World Models for Parallel Code [38] — predicts race conditions and profiling artifacts from parallel source.
- Industrial CWM / InCoder-32B-Thinking [37] — CWM recipe on Verilog and GPU execution traces.
- The Double Life of Code World Models [58] — CWM trace predictions repurposed for malicious-behavior detection.
- Towards a Neural Debugger for Python [59] — neural debugger as forward/inverse world model.
- Neural Computers [60] — video-model-style WMs of CLI/GUI runtime from I/O traces.
- Generating Code World Models with LLMs Guided by MCTS [9] — the WM is *the code itself*, synthesized by an LLM.
- General Agents Contain World Models [4] — proves that sufficiently competent goal-conditioned agents must contain extractable world models, under restrictive conditions discussed critically in §17.

0.9.7 7.7 GIF-MCTS / Generating Code World Models via MCTS [9]

GIF-MCTS treats the world model itself as a Python program — an `Environment.step(s,a) → (s', r, done)` class synthesized by an LLM to match a small batch of pre-collected `(s, a, r, s', d)` transitions. MCTS over partial programs uses three action types (*generate* lines, *improve* full program given a failing transition, *fix* runtime/syntax errors), with reward equal to the fraction of transitions reproduced correctly. The synthesized world model, once compiled, runs 4–6 orders of magnitude faster than calling an LLM as world model. On APPS-Competition it reaches 28.3% strict pass@20 (Llama-3-70B), beating WorldCoder's 25.1%. The conceptual contribution is to *search over candidate Python world-model programs* rather than train a neural one.

0.10 8. World Models for Code Agents

Once an LLM is an *agent* taking actions in a non-trivial environment, the world-model question becomes whether the agent simulates the environment's response. Three sub-environments dominate.

0.10.1 8.1 Web agents

Web Agents with World Models [61] systematizes the thread. DyMo / World Modeling Improves LM Agents [45] adds a next-state prediction head to function-calling agents and reports gains on BFCL-V2 — though with a caveat (§17): the WM head reaches 90–94% state-prediction accuracy while the underlying policy reaches only 72.8% task success, illustrating that WM-head accuracy and agent accuracy can decouple.

WebDreamer [31]. WebDreamer treats the web as a POMDP in which the LLM imagines natural-language state-change descriptions for each candidate click, type, or select. A specialist *Dreamer-7B* (Qwen2-VL-7B fine-tuned on 3.1M synthesized (initial visual state, action, state-change) tuples from random walks over Common Crawl URLs) provides cheap rollouts. At inference, model-predictive control samples actions, scores simulated trajectories with GPT-4o on a 3-scale rubric, and executes the argmax. On VisualWebArena, Online-Mind2Web, and Mind2Web-Live, this beats the reactive baseline by +34/+42/+24% relative (?+6–11 absolute) while running 4–5× faster than tree search. The bet: when actions are irreversible (forms, purchases), one-step MPC over an LLM-as-WM beats backtracking search.

0.10.2 8.2 OS / computer-use agents

Reinforcement World Model Learning for LLM-based Agents [39] and World Models as an Intermediary between Agents and the Real World [62] generalize the lens: a learned WM mediates between LLM and expensive environment.

Dyna-Think [63]. Dyna-Think trains a single Qwen2.5-32B to internalize world-model simulation inside its `<think>` block for OSWorld and WindowsAgentArena. Two stages: DIT (imitation learning on R1 traces cleaned to keep only WM-simulation text) and DDT (Dyna-Q-style joint training over three WM heads — next state, state-diff, critic-prediction — with rejection-sampled policy updates). On OSWorld BoN the 32B model reaches 43.1, essentially matching DeepSeek-R1 at 685B with half the tokens. World-model accuracy correlates with task success at r=0.32 across models. This is the cleanest instance in the corpus of policy and learned world model hosted in the *same* LLM.

0.10.3 8.3 SWE agents

SWE-bench [26] and SWE-Gym [64] defined the eval and training environment respectively. CodeAct [27] made the Python interpreter the unified action space. Reflexion [22] was the earliest entry with episodic verbal RL. Nanbeige SWE-World [65] trains a learned Docker-free execution surrogate. Understanding by Reconstruction [66] reverses the development process to harvest agentic pretraining traces. SWE-TRACE [67] provides process-level reward modeling over trajectories. Self-Play SWE-RL [68] introduces adversarial bug-injection/repair self-play. Bootstrapping Coding Agents — The Specification Is the Program [42] reframes the SWE task itself as a programmatic spec.

The §16 empirical synthesis separates three regimes: execution-grounded open-weight model training (CWM, SWE-RL), execution-grounded agents with learned simulators (Nanbeige SWE-World), and scaffold evolution around closed-model executors (Darwin GM, Huxley GM). Under best-of-k or verifier-reranked protocols, several systems report 60–68% on SWE-bench Verified, but those numbers are not directly comparable to pass@1 model-training results, and the scaffold-evolved systems are not "32B open-weight world-model-trained" — they run frontier closed models inside an evolved harness.

0.11 9. RL with Execution as the World Signal

The model-based-RL framing — world model is what the policy plans over — has produced a clean lineage.

0.11.1 9.1 RLEF [30]

RLEF formulates iterative code synthesis as a POMDP — actions are full code responses, observations are formatted public-test execution feedback, rewards come from held-out *private* tests. Standard PPO with KL regularization and a 3-turn limit. Llama-3.1-70B+RLEF reaches 37.5/40.1 pass@1 valid/test on CodeContests at budget 1@3 (vs 25.9/27.5 baseline), matching AlphaCodium-GPT-4 with 5 samples; at 10@100 it reaches 54.5/54.5, surpassing the AlphaCode 41B+clustering baseline. Critically, a random-feedback ablation removes the entire gain, isolating that the model is learning to *use* execution feedback rather than just sample more.

0.11.2 9.2 SWE-RL [33]

SWE-RL applies GRPO to 273k high-quality PR seeds with a rule-based, continuous reward (`difflib.SequenceMatcher` similarity between predicted and oracle patch) — *no code execution at training time*. Llama-3.3-70B fine-tuned this way (Llama3-SWE-RL-70B) hits 41.0% pass@1 on SWE-bench Verified with the Agentless Mini scaffold. The surprising result is OOD transfer: HumanEval+ 76.2?79.9, CRUXEval-O 61.9?75.5, MATH 70.9?73.7, while SFT on the same data *degrades* on these. Continuous reward beats discrete in ablation (34.8 vs 29.0 oracle-repair). The thesis: partial-credit similarity rewards on real PR patches induce reasoning patterns that transfer beyond the training distribution.

0.11.3 9.3 Process Reward Models

ExecVerify [51], SWE-PRM [52], DataPRM [69], ThinkPRM [53] form a cluster where the WM is a learned *evaluator* of partial trajectories. As §17 develops critically, this is not the same object as a forward world model — PRMs are critics with execution grounding. They cannot roll out, cannot simulate counterfactuals. Survey hygiene argues for keeping the distinction.

0.12 10. Planning and Search with Code World Models

0.12.1 10.1 RAP [25]

RAP frames reasoning as MCTS in a self-consistent MDP where the same frozen LLM serves as both policy and transition model — a state is a textual configuration (blocks layout, intermediate variables, current fact), an action is a step proposed by the LLM, and the transition is obtained by re-prompting. Rewards combine action likelihood, state confidence (majority voting), self-evaluation ("Is this correct?"), and task heuristics. On Blocksworld 4-step, RAP@10 reaches 0.86 with LLaMA-33B, surpassing GPT-4+CoT's 0.63 by 33% relative. The conceptual template — *repurpose the LLM as both policy and transition model under MCTS* — is what every later LLM-as-WM paper extends.

Tree of Thoughts [70], AlphaZero-like Tree Search for LLM Decoding [71], Tree Search for LM Agents [72], and Mastering Board Games by External/Internal Planning with LMs [73] develop the search frame; the last gives the clearest contemporary recipe for learned tree-search with LLM-as-WM, straightforwardly transferable to code.

0.12.2 10.2 Execution-conditioned generation

Execution Guided Line-by-Line Code Generation [74] uses classifier-free guidance to condition next-token prediction on candidate-runtime outcomes. Jupiter [75] formulates notebook state as MCTS nodes. REPL-Plan [76] reuses a REPL state pool across tasks. Substrate is well-developed for short-horizon code-gen; less so for long-horizon multi-file SWE.

0.13 11. JEPA, Dreamer, and the Latent-Action Gap

LeCun's Joint Embedding Predictive Architecture (I-JEPA, [77]) predicts in embedding space rather than pixel space. The Dreamer family — Hafner et al.'s DreamerV1 [78], DreamerV2 [79], and DreamerV3 [80], built around the Recurrent State-Space Model — has near-zero direct application to code. Two papers occupy the gap.

0.13.1 11.1 LLM-JEPA [34]

LLM-JEPA adds a joint-embedding predictive objective to standard NTP training, using (text, code) as the two JEPA views with the LLM's last-layer last-token hidden state as encoder and a tied-weights `[PRED]` token as predictor. The loss is `L_NTP(text) + λ · d(Pred(Enc(Text)), Enc(Code))` with cosine distance. On Llama-3.2-1B fine-tuned on NL-RX-SYNTH the gain is 57.3 ? 71.5 (+14.2 absolute); on Spider, GSM8K, and HellaSwag the wins are smaller. The top-100 singular values of `Enc(Text) − Enc(Code)` collapse by orders of magnitude, indicating a low-rank text?code mapping. Whether this is "JEPA in the LeCun sense" or a regularizer on top of NTP is the live question raised in §17.

0.13.2 11.2 CoLA [8]

CoLA replaces the 128k-token action space of an LLM with a small learned latent-action codebook. Three modules: a VQ-VAE-style inverse-dynamics model that infers latent action a_t from $(x_{1:t}, x_{t+1})$; a language world model that inserts the chosen latent action into the

LLM embedding stream and decodes the next token; and a policy π(a□ | x₁:t) behavior-cloned from inverse-dynamics labels then RL-tuned. Action-level MCTS over the learned codebook (with a Double-DQN Q-function) reaches Math-500 68.2 vs 63.0 baseline MCTS-Q. CoLA is the corpus's clearest "Dreamer-for-LLMs" instance — the action space is genuinely compressed, and rollout/search operate in that compressed space.

0.13.3 11.3 The gap

Despite CWM and dozens of LLM-as-world-model papers, *no public Dreamer/RSSM-style latent-imagination world model has been trained for SWE agents.* CWM rolls out in token space. CoLA is the closest concrete instance. UniZero [81] generalizes MuZero with transformers but is rarely instantiated on code. Genie [82] gives the vision-side template. JEPA for RL [83] extends the energy-based objective to RL.

Whether the gap matters is a live question developed critically in §17. The vision-domain pressure that motivated Dreamer's RSSM design (pixel-space rollout cost) does not exist for code, where state is small and the simulator is available. The argument *for* latent imagination rests on inference-time speed and the action-space compression CoLA demonstrates, not on rollout cost per se.

0.14 12. Specialized Domains

Diffusion code models. DiffuCoder [40], Dream-Coder 7B [41]. Iterative denoising naturally accommodates plan-then-refine generation.

Decompilation and cross-language. SK2Decompile [84], SALT4Decompile [85]. Translation as semantic-simulation task. EquiBench [86] supplies the equivalence eval.

Hardware / RTL. VeriRL [87], ChipSeek [88], VeriCoder [89] form a cluster where the simulator is the world model. Hardware is an attractive domain because simulators are precise, fast, and deterministic — closer to Atari than to Python.

ARC and abstract synthesis. Executable World Models for ARC-AGI-3 [10] instantiates literal-WM-per-task: synthesize a Python world model verified against observations. SOAR [90] evolves programs over ARC. Darwin / Huxley Godel Machines ([91], [92]) close the self-improvement loop.

0.15 13. Reasoning, Process Rewards, Memory

Long-CoT reasoning for code. o1-Coder [93] replicates o1 with MCTS+RL. R1-Code-Interpreter [94] supplies the open SFT+RL recipe across 144 tasks. Scaling Test-Time Compute to Achieve IOI Gold Medal [95] shows open-weight gpt-oss-120b matching closed reasoning models via inference-time scaling.

Long-CoT reasoning is *mental execution* — the chain-of-thought simulates the world model the network never explicitly trained. CWM-style explicit world modeling and R1-style reasoning are partial substitutes; whether they compose multiplicatively is open.

Memory. Episodic Memory is the Missing Piece for Long-Term LLM Agents [96] frames the gap. Memory as Action [97] treats memory operations as RL-learnable actions. RepoGraph [44] provides a durable repo-level dependency graph.

0.16 14. Verification, Probing, Safety

0.16.1 14.1 Formal verification: the leading edge

The verifier-grounded lineage is the only research direction in the corpus that does not rely on LLM self-report for correctness — the verifier provides ground truth.

ATLAS [46] is the cleanest end-to-end pipeline in the corpus for scaling verified-code data. From TACO-verified (12.8k LeetCode-style problems with Python references and tests) ATLAS produces 2,751 verified Dafny programs decomposed into 19,385 training examples across six tasks (NL-to-Code, NL-to-Spec, Spec-to-Code, Spec-Repair, Impl-Repair, Proof-Infilling). Spec quality is filtered by three SMT-discharged lemma types — soundness, completeness-contradiction, and completeness-perturbation. Qwen-2.5-Coder-7B fine-tuned this way reaches DafnyBench Pass@1 of 55.8% (from 32.4%) and DafnySynthesis Pass@5 of 65.8% (from 15.8%, surpassing GPT-4's 53.4). ATLAS does for verified Dafny what auto-formalization did for Lean theorems.

The cluster also includes Re:Form ([47], Dafny+RL), CLEVER ([48], Lean), VeriStruct ([49], Verus), AutoRocq ([50], Rocq), and Semantic Equivalence Self-Play with Formal Verification ([98], Liquid Haskell).

CLEVER's 1/161 end-to-end Lean result is the most sobering number in the corpus: frontier models, with Lean type-checker access for self-verification, still fail on >99% of HumanEval-derived problems requiring joint spec + implementation verification. Understanding Formal Reasoning Failures in LLMs as Abstract Interpreters [99] is the diagnostic: when asked to reason in the style of formal abstract interpretation over 22 SV-COMP programs, all frontier reasoning models make systematic errors in widening, fixpoint termination, and join operations.

0.16.2 14.2 Symbolic execution and LLMs

AutoBug [100], SESpec [101], LLM-Sym [102], Loop Invariant Generation via Reasoning LLMs + SMT [103] combine LLMs with concrete or symbolic engines. The unifying pattern: the LLM hypothesizes the world model; symbolic execution verifies or extends it.

0.16.3 14.3 Probing and mechanistic interpretability

Mechanistic Interpretability of Code Correctness via SAEs [104] and On LLMs' Internal Representation of Code Correctness [105] ask what code LLMs actually represent. Findings: *partial, brittle* internal execution representations — a vindication of explicit trace pretraining.

0.16.4 14.4 Repair and debugging as world-model probing

Self-Debug [23], InspectCoder [106], Agent That Debugs — Dynamic State-Guided Vulnerability Repair [107], Agentic Code Reasoning ([108], semi-formal execution-path reasoning without running code). Shared pattern: maintain a belief over program state, query the runtime to update the belief, act on the posterior — Bayesian world-modeling in everything but name.

0.16.5 14.5 Safety and malicious code

The Double Life of Code World Models [58] repurposes CWM-style trace predictions for malicious-behavior detection. CodeBreaker [109] is the offensive analogue. Concolic Execution + LLM for Zero-Day Malware Detection [110] pairs path-prioritization with concrete execution.

0.17 15. Benchmarks and the Evaluation Gap

Benchmarks split cleanly by what they measure.

- Static code quality — HumanEval, MBPP, LiveCodeBench [111] measure code-LLM output without exercising the world-model claim.
- Execution reasoning — CRUXEval [21], REval [54], CRUXEval-X [112], TraceEval [113], PLSemanticsBench [114]. The canonical WM evals.

- Semantic equivalence — EquiBench [86], CodeARC [115].
- Agentic — SWE-bench, SWE-Gym, WebArena [116], Mind2Web [117], PyBench [118], OSWorld, WindowsAgentArena.

The eval gap. No widely adopted benchmark directly measures world-model fidelity by holding the policy fixed and varying the WM. Every measurement of WM capability is mediated through downstream task performance, so we cannot distinguish *the model has internalized program semantics* from *the model is exploiting trace-token shortcuts in the test distribution*. Demystifying Errors in LLM Reasoning Traces [36] supplies the diagnostic: even DeepSeek-R1, o4-mini, Gemini 2.5 Flash, and Claude 4, when asked to simulate execution and explain their reasoning, produce traces with errors clustering into nine categories (Computation, Indexing, Control Flow, Skip Statements, Misvaluation of Native API, Hallucination, Input Misread, etc.). Models with 85–98% final-answer accuracy on output prediction produce traces with systematic errors throughout. The decoupling — high outcome accuracy, low process fidelity — is the signature of a system that has learned to predict outcomes without faithfully simulating dynamics.

0.18 16. Empirical Landscape

0.18.1 16.1 SWE-bench scoreboard

Protocols mix in SWE-bench reporting and are non-comparable across the obvious axes. We split the table into three protocol classes and ask readers to compare within, not across, classes. Training / Evaluation regime: *Frozen-model scaffold* = no model training (agent loop around a frozen closed model); *EG-LLM SFT* = supervised fine-tune on agent trajectories; *EG-LLM + RL* = RL with execution rewards; *EG-agent + learned simulator* = SFT+RL plus a learned execution surrogate; *EG-LLM + trace mid-train + RL* = the CWM lineage; *Scaffold-evolved* = recursive scaffold/agent self-improvement around a frozen closed model. WM-Arch is reserved for systems with explicit transition/rollout machinery (N1/N2/N3 from §3); no row in this table qualifies.

Table 16.1a — Single-sample resolution (pass@1 on Verified unless noted)

System	Base model	Bench	Pass@1	Regime	Source
SWE-agent	GPT-4 Turbo	full	12.5%	Frozen-model scaffold	[28]
AutoCodeRover	GPT-4	Lite	19.0%	Frozen-model scaffold	[119]
Agentless	GPT-4o	Lite	32.0%	Frozen-model scaffold	[120]
SWE-Gym	Qwen-2.5-Coder-32B	Verified	20.6%	EG-LLM SFT	[64]
Agent-RLVR (no RM)	Qwen-2.5-72B	Verified	22.4%	EG-LLM + RL	[121]
Long-Context Multi-Turn RL	Qwen-2.5-72B	Verified	39.0%	EG-LLM + RL	[122]
SWE-RL (Llama3-SWE-RL-70B)	Llama-3.3-70B	Verified	41.0%	EG-LLM + RL	[33]
Nanbeige SWE-World (RL)	Qwen-2.5-Coder-32B	Verified	55.0%	EG-agent + learned simulator	[65]

System	Base model	Bench	Pass@1	Regime	Source
CWM	CWM-32B	Verified	*not directly reported* †	EG-LLM + trace mid-train + RL	[2]

† The CWM paper does not report a pure pass@1 number under the standard protocol; the headline 65.8% in Table 16.1b is best@16 with verifier reranking. Inferred pass@1 from the paper's reported figures is approximately 53–55%, but this is *our estimate, not a directly reported number*, and we do not place it in the main row.

Table 16.1b — Best-of-k with verifier reranking / TTS

System	Base	Bench	Score	Sample budget	Source
Agent-RLVR + RM rerank	Qwen-2.5-72B	Verified	27.8%	rerank over multi-sample	[121]
Nanbeige SWE-World (TTS@8)	Qwen-2.5-Coder-32B	Verified	68.2%	best@8	[65]
CWM (TTS)	CWM-32B	Verified	65.8%	best@16 over 40 reranked	[2]

Table 16.1c — Scaffold-evolution and self-play around closed-model executors

System	Backbone executor	Bench	Score (start → end)	Note	Source
Darwin Godel Machine	Claude-3.5-Sonnet	Verified	20% → 50%	Scaffold evolves over 80 iterations	[91]
Huxley Godel Machine	GPT-5-mini	Verified (500)	— → 61.4%	Scaffold optimized for Verified-60	[92]
SICA	Claude-3.5-Sonnet + o3-mini	Verified (subset)	17% → 53%	Scaffold evolves; not a model-training method	[123]

Citations that quote "CWM 65.8% SWE-bench Verified" without disclosing the best@16/TTS protocol should be read as misreporting, not as direct pass@1 numbers.

What can and cannot be compared. Within Table 16.1a the comparable axis is pass@1. SWE-RL's 41.0%, Long-Context-MT-RL's 39.0%, and Nanbeige SWE-World's 55.0% are roughly comparable as "open-weight WM-trained pass@1 on Verified." Table 16.1c sits in a different category — these are scaffold-evolution methods over closed models, and treating them as evidence for "world-model training works" conflates scaffold search with model improvement. Reading them inside Table 16.1a, as some online discussion does, is apples-to-oranges.

Two empirical claims, separately defensible. (i) On *pass@1*, open-weight WM-trained 32B systems (Nanbeige 55.0%, Long-Context-MT-RL 39.0%, SWE-RL 41.0%) outperform frozen open-weight baselines (Qwen-2.5-Coder-32B raw 6.2%) by 30–50 absolute points; this gain is the strongest case for training on agent trajectories with execution feedback. (ii) On *best@k with verifier reranking* (Table 16.1b), the same models reach 65–68%, but the additional 13–15 points are attributable to TTS reranking infrastructure, not to the WM training. Conflating (i) and

(ii) by quoting CWM's 65.8% alongside SWE-RL's 41.0% as both "world-model-trained pass@1 SOTA" is exactly the misreporting the protocol split is designed to prevent.

0.18.2 16.2 Trace pretraining gains on execution-reasoning

Paper	Backbone	Baseline	After trace pretrain/FT	Delta	Benchmark
TRACED	UnixCoder	—	+12.4% rel branch-coverage; +25.2% rel variable-value	rel	CodeNet exec
NExT	PaLM 2-L	23.2	49.3	+26.1 abs	MBPP-R
NExT	PaLM 2-L	32.2	42.5	+10.3 abs	HumanEvalFix-Plus
SemCoder (1.3B)	DS-Coder 1.3B	base	63.6 / 63.9	+23 abs	CRUXEval-I / O
"What I cannot execute"	Llama-3.1-8B	37.8%	~80%	+42 abs	CRUXEval-O
Do Code Semantics Help?	DSCoder, Llama-3, Gemma-2	various	? a few abs; some regressions	mixed	comprehensive

For under-trained ?8B open-weights, trace pretraining delivers +15 to +42 absolute on CRUXEval-O. The "Do Code Semantics Help?" ablation [3] is the disconfirming evidence: across multiple backbones and five trace representations, no single representation consistently outperforms others, and several downstream tasks regress under trace augmentation. The gain shrinks rapidly with base-model quality. Trace pretraining is a remedial intervention for weak code models; whether frontier models still benefit is unsettled.

0.18.3 16.3 Web/OS agents from WMs

Paper	Benchmark	Base	With WM	Delta	Mechanism
WebDreamer (GPT-4o WM)	VisualWebArena	17.6%	23.6%	+34.1% rel (?+6 abs)	LLM-as-WM + MPC
WebDreamer	Online-Mind2Web	26.0%	37.0%	+42.3% rel (?+11 abs)	same
Dreamer-7B (trained WM)	VisualWebArena	base	+4.7 abs	—	trained WM
WMA [61]	WebArena	base	action-selection 52?70%	—	trained transition WM
Dyna-Think DDT (32B)	OSWorld BoN	RFT~28%	43.1%	?+15 abs	Dyna-Q + WM head
Dyna-Think DDT	WindowsAgentArena	28.4%	34.9%	+6.5 abs	same

WM gains on web/OS are real but quantitatively small (?+5–10 absolute task success rate on most benchmarks), and partly confounded with the extra synthetic data the WM generates. DyMo's 90%+ state-prediction accuracy versus 72.8% task success rate exemplifies the decoupling: WM heads can be accurate without the agent being accurate.

0.18.4 16.4 Formal verification vs LLM-only

System	Language	Benchmark	Baseline	With system	Source
ATLAS	Dafny	DafnyBench Pass@1	32.4%	55.8%	[46]
ATLAS	Dafny	DafnySynthesis Pass@5	15.8%	65.8% (>GPT-4 53.4)	[46]
CLEVER	Lean 4	161 problems end-to-end	best frontier	?1/161	[48]
VeriStruct	Verus / Rust	11 modules	—	99.2% (128/129 fns)	[49]
AutoRocq	Rocq	math + verif lemmas	5 baselines	48.0% math / 30.9% verif	[50]
Semantic Equiv Self-Play	Liquid Haskell	EquiBench	base	+13.3 pp	[98]

Verified codegen has the steepest training-data sensitivity in the survey: small synthetic datasets (2.7K verified Dafny programs in ATLAS) produce +25–50 absolute gains because LLM-only baselines start near zero. CLEVER's ?1/161 shows that without explicit data/scaffolding, frontier models cannot reliably produce verified code. VeriStruct shows that on curated targets near-perfect is reachable.

0.18.5 16.5 Reasoning-model competitive programming

Model	Codeforces	IOI / ICPC
gpt-4o	808 (11th pct)	—
o1-preview	1258 (62nd pct)	—
o1	1673 (89th pct)	—
o1-ioi	1807 (~93rd pct)	IOI 2024 49th pct live
o3	elite-human-class	IOI 2024 gold
gpt-oss-120b + GenCluster	—	IOI 2025 gold (open-weight)
Gemini 2.5 Pro Exp	—	ICPC-Eval Pass@1 22.0%
DeepSeek-R1	—	ICPC-Eval Pass@1 14.4%
Claude 3.7 Sonnet	—	ICPC-Eval Pass@1 11.8%
GPT-4o	—	ICPC-Eval Pass@1 5.9%

Codeforces +999 rating points in 14 months on the o-series. The same line shows that frontier reasoning models gain almost everything from RL scaling, not from domain-specialized scaffolds — which complicates the case that "trace-style world models" are doing causal work for frontier systems.

0.19 17. Critical Perspectives

This section names where the field overclaims, where the consensus is fragile, and where vocabulary is doing more work than evidence. We develop seven theses.

0.19.1 17.1 The "world model" label has become marketing for any code LLM trained on something other than raw source

This is a terminology argument, not a contradiction with §3. §3 deliberately admits "implicit WMs in token policies" as a fourth flavor — including CWM, TRACED, SemCoder under that

flavor — because that *is* how the field currently uses the term. The claim of §17.1 is that this permissive definition is what allows the rhetorical sleight of hand, and that a stricter definition would be more useful going forward.

Concretely: read CWM [2] carefully and the architecture it ships is a 32B decoder-only Transformer with GQA, sliding-window blocks, RoPE, AdamW — a Llama-class model. What earns it the "world model" badge is the mid-training datamix: 5T tokens of Python observation-action traces plus ForagerAgent SWE trajectories. There is no separate dynamics head, no inverse model, no recurrent latent. The same observation lands on LLM-JEPA, DyMo, and most "world model" papers from 2024–2026: the artifact is a standard LLM with an enriched objective.

A useful purity test: *can we ablate the supposed world-modeling component without changing the architecture?* If yes, the system is a trace-trained LLM. If no, there is a genuine architectural commitment. By that test, CoLA [8] and the Dreamer-for-LLMs gestures pass. CWM, SemCoder, NExT, and most of the "explicit WM" cluster fail. We propose that "world model" be reserved for systems with the architectural commitment, and that *execution-grounded code LLM* serve for the rest. The §3 permissive definition can stay as descriptive — what the field currently calls a code WM — but a normative tightening is warranted, and the survey from §17 onward uses the stricter sense.

0.19.2 17.2 Trace pretraining has a causal-isolation problem the surface numbers obscure

"Do Code Semantics Help?" [3] is the most damaging paper for the prevailing optimism. It runs a comprehensive ablation across DeepSeek-Coder, LLaMA-3, and Gemma-2 with five representations (Scratchpad, NExT, CodeExecutor, Concise, SemCoder) on program repair, code synthesis, BigCodeBench, LiveCodeBench, and CRUXEval. Its headline: integrating trace-based semantic information into SFT *cannot significantly enhance* code-generation ability. In 7 of 9 synthesis settings the no-trace baseline wins or ties. At inference, in 36 of 56 test-scaling configurations, trace prompts hurt.

"What I cannot execute, I do not understand" [55] is gentler — Execution Tuning reaches ~80% CRUXEval-O — but the same paper's downstream evaluations on HumanEval, MBPP, and GSM8K show *negligible* gains from trace data in the SFT mix.

The strongest counterexample to "barely transfers" is NExT [29], which pushes PaLM 2-L on Mbpp-R from 23.2% to 40.8% *with traces removed at test time* — a +17.6 absolute gain on the no-trace-at-inference setting, which is exactly the transfer pattern the skeptical reading says shouldn't happen. The honest version of the thesis is therefore narrower: trace pretraining transfers to *program-repair* tasks where the model needs to reason about a buggy program's behavior (where NExT and TraceFixer-style work shines), and barely transfers to *fresh code synthesis* on benchmarks like HumanEval and MBPP that are closer to the model's prior. The "Do Code Semantics Help?" disconfirmation is on synthesis; NExT's transfer is on repair. Both can be true.

The honest reading: trace pretraining helps execution prediction (the thing trained on), transfers to runtime-reasoning-heavy downstream tasks like repair, and barely transfers to fresh code synthesis, exactly the pattern you would expect if dense execution supervision is teaching a runtime-tracking skill rather than a generative model of program intent.

CWM's 65.8% on SWE-bench is the apparent counterexample, but the Meta team reports it after trace mid-training *and* 3M ForagerAgent SWE trajectories *and* multi-task RL with verifiable rewards — three interventions stacked. The "world model" component is not causally isolated from the SWE-trajectory and RL components. Without an ablation that removes trace mid-training while holding ForagerAgent and RL fixed, the headline is unfalsifiable. The field has agreed to call CWM a world-model success because the name is on the model card, not because the experimental design demonstrates it.

0.19.3 17.3 The Dreamer-for-code gap may be a non-problem

The conventional framing treats the absence of latent-imagination world models for SWE as the field's largest architectural gap. The empirical record argues the opposite. CWM in token space

reaches 65.8% SWE-bench. CoLA produces respectable but not field-shifting results, and even there the WM is fine-tuned on top of a standard LLM rather than replacing it.

Vision world models needed latent rollouts because pixel-space rollouts were too expensive — a frame is ~10^6 dimensions, dynamics are partially observed. Program execution is the opposite: a Python frame is small, dynamics are observable, and the simulator (CPython) is available for free at training time. The pressure that drove Dreamer's RSSM design does not exist for code.

Debugging Code World Models [35] shows CWM's long-horizon failures are dominated by *action hallucination*, not state-propagation error — under teacher forcing CWM tracks state correctly for 128 steps. A latent rollout would compress states but not fix the action policy, which is the actual bottleneck.

The counterargument is fair: latent rollouts permit faster planning at inference, and for multi-agent or population-scale search the speedup is asymptotically meaningful. But the survey should retire "single largest architectural gap" framing and replace it with "an interesting open question whose payoff is not yet demonstrated."

0.19.4 17.4 PRMs are critics, not world models

Process reward models — ExecVerify [51], SWE-PRM [52], ThinkPRM [53], FunPRM [124], DataPRM [69] — are often grouped under the world-modeling umbrella. This is wrong in a way that matters. A world model is, by every definition the survey uses, a *forward* predictor of `(state, action) → next_state`. A PRM is a *backward-looking evaluator*: given a partial trajectory, score it. In classical model-based RL these are different objects — Dreamer has both a world model (RSSM) and a critic (value function).

PRMs cannot roll out. Cannot simulate counterfactuals. Cannot be used by a planner that wants to score a hypothesized future. Conflating them dilutes the world-model concept until it means "any neural network trained on execution-related signals" — at which point the term is useless. Vocabulary discipline is cheap and the field would benefit from it.

The same critique applies, less severely, to verifier-grounded systems: a Lean proof checker is not a world model, it is a deterministic verifier of a candidate output. Calling it "the world model" when ATLAS, Re:Form, or AutoRocq use it makes for a tidy survey arc but blurs the actual computational structure.

0.19.5 17.5 "General Agents Contain World Models" is much weaker than its title suggests

Richens et al. [4] prove that any goal-conditioned policy satisfying a regret bound δ for sufficiently deep composite goals (depth $n \square 1$) must encode an extractable approximation of the transition function with bounded error. Genuine and elegant.

But read the assumptions: fully observed environment, finite communicating stationary controlled MDP, goal-conditioned policy satisfying a regret bound for a specific class of LTL composite goals of depth n. Theorem 2 of the same paper explicitly shows that for myopic agents (depth-1 goals), *no world model is needed*. Real SWE agents are myopic-ish over short turns and approximately competent over longer ones; their environments are partially observed (rarely full filesystem state); they violate stationarity (the repository changes under their actions); and their regret bound for arbitrary composite goals is unknown and almost certainly not satisfied. The authors caveat this in §6 ("Limitations") of their paper.

The theorem is a beautiful existence proof for an idealized agent class. It is *not* an empirical statement that SWE coding agents have learned world models, and it provides no guidance about the fidelity of any world model they may have learned.

0.19.6 17.6 The verifier-grounded lineage is the actual leading edge

ATLAS, Re:Form, CLEVER, VeriStruct, AutoRocq, and the Liquid Haskell self-play paper [98] share a property no LLM-only system possesses: code whose correctness is *machine-checked* against a formal specification. Compare to the SWE-bench paradigm, where "correctness" means "hidden unit tests pass" — a weaker guarantee, since unit tests cover specific inputs and the system can pass them while being wrong on adjacent inputs.

The abstract-interpreter paper [99] is the diagnostic: when frontier reasoning LLMs are asked to reason in the style of formal abstract interpretation over 22 SV-COMP programs, they make systematic errors in widening, fixpoint termination, control-flow propagation, and meet/join operations. They generated unsound invariants on programs as small as `count_by_2.c`. If LLMs cannot reliably perform interval-domain abstract interpretation on toy C programs, claims that they have learned faithful internal world models of program semantics are doing a lot of inferential work.

The verifier-grounded line is the only research direction that does not rely on LLM self-report for correctness. Scoped carefully, it leads on *synthesis-from-spec* (ATLAS reaches 65.8% Pass@5 on DafnySynthesis at 7B, surpassing GPT-4) and on *near-perfect verified completion of curated targets* (VeriStruct 99.2% on 11 Verus modules). It does *not* lead on end-to-end verified codegen from natural language: CLEVER reports □1/161 on Lean problems requiring joint spec + implementation verification. The future of *correct* code is plausibly hybrid — neural proposal, symbolic verification, with the verifier providing ground truth that learned world models cannot — but the hybrid is not yet a finished pillar. We catalog the verifier line in §14.1 alongside symbolic execution and abstract-interpretation probing; readers who accept this thesis should weight that subsection more heavily than its sequence position suggests.

0.19.7 17.7 The evaluation gap is the structural reason the field looks confused

Across CRUXEval, REval, CRUXEval-X, PLSemanticsBench, TraceEval, and EquiBench, no benchmark holds policy fixed and varies world-model quality. Every measurement of "world-modeling capability" is mediated through downstream task performance, so we cannot distinguish *the model has internalized program semantics* from *the model is exploiting trace-token shortcuts in the test distribution*.

"Demystifying Errors in LLM Reasoning Traces" [36] is the diagnostic: even DeepSeek-R1, o4-mini, Gemini 2.5 Flash, and Claude 4, when asked to simulate execution, produce traces with errors in nine systematic categories. Models with 85–98% final-answer accuracy on output prediction produce traces with systematic errors throughout — high outcome accuracy, low process fidelity, the signature of a system that predicts outcomes without faithfully simulating dynamics.

Self-repair literature exhibits the same pathology: Olausson et al. [125] showed that GPT-4 self-repair on APPS and HumanEval, normalized by compute, often performs *worse* than i.i.d. resampling. The bottleneck is the model's feedback quality, not its repair capability — human-written feedback boosts repair success by 1.58×. The model can generate code, can sometimes recognize bugs, but cannot reliably simulate why its code is wrong — which is exactly what a faithful world model would let it do. The empirical bound on LLM self-repair is, in effect, an empirical bound on the fidelity of the implicit world model the LLM is running. Calling that world model internal is fine; calling it good is not.

Until benchmarks measure process fidelity independently of outcome, "is this system actually building a world model?" remains scientifically undecidable.

0.20 18. Open Problems

The critical perspectives of §17 reshape the conventional open-problems list. We propose six problems where the literature is thinnest *and* the upside is largest.

1. Causal isolation of trace-pretraining contributions. Every claim of the form "this WM-trained model achieves X" should be paired with an ablation removing the WM component while holding training data and RL fixed. CWM in particular needs this. Without it, the headline numbers underdetermine whether the WM did the work.

2. World-model fidelity as a first-class metric. §15's eval gap is concrete: build a benchmark where holding policy fixed and varying WM quality causes measurable variation in planning quality, independent of downstream task. This benchmark would clarify the field more than any single new model.

3. Hybrid neural-symbolic systems. §17.6 argues the verifier-grounded line is the leading edge. The natural integration is *neural proposal, symbolic verification*, with the verifier providing gradient-free correctness signal and the neural component providing proposals at scale. Differentiable surrogates of symbolic verifiers (Lean / Dafny / Rocq) that pass verifier-style gradients during training are open.

4. Multi-modal WMs for coding. GUI agents need pixel-level WMs (Neural Computers, [60], is a first attempt). Tying pixel WMs to code-state WMs through a shared latent is essentially unsolved.

5. Long-horizon credit assignment with execution-grounded rewards. PRMs (§9.3) are early, and §17.4 argues they should be conceptually separated from world models. The right structure for rewarding an agent across hundreds of execution-grounded steps is a live question.

6. World models of the developer, not just the program. All current WMs model the *machine*. Few model the *developer intent* with comparable fidelity. ATLAS and Re:Form gesture in this direction by treating the spec as the WM. A full developer-intent WM would close the agentic loop.

We do not list "Dreamer-for-SWE-agents" as the field's largest gap, contrary to common framing. §17.3 argues the pressure motivating that direction in vision does not transfer to code. It remains an interesting research question, not the highest-leverage one.

Three under-explored representations. The §5.3 taxonomy table flags three classes of representation, mature in vision-WM, that have no code-WM exemplar:

- *Global Latent Vector (Dreamer-style RSSM)* — discussed and contested above. The Hafner et al. DreamerV1–V3 line proves the design space exists; whether it pays off for code is the question §17.3 leaves open.
- *Spatial / Structural Grid* — the analog of OccWorld / BEV for code would be a learned predictive grid over AST nodes, call-graph edges, or CFG states. RepoGraph [44] shows the static version is useful as agent state; the predictive version is unexplored.
- *Decomposed Object / Slot (object-centric WMs)* — the analog for code would model variables, scopes, or classes as discrete persistent slots whose state propagates independently. No paper in the corpus instantiates this, despite obvious mappings (each variable is an object, each frame is a scene).

The object-centric and structural-grid gaps look more genuine than the Dreamer one, in our reading, because they exploit structure that code *already has* (objects = variables, grids = AST/CFG) rather than borrowing pressure from a domain (vision) where the structural assumptions differ.

0.21 19. Conclusion

Across the literature surveyed here, a single trajectory is visible: from neural execution (modeling the machine), through trace pretraining (modeling execution implicitly), to CWM and its descendants (modeling execution explicitly with a named artifact), to agentic SWE and RL (modeling the environment), to JEPA and latent-action models (modeling in compressed space), and on toward formal verification, probing, and safety (modeling reliably). What was a scattered set of insights in 2014 has by 2026 cohered into a recognizable research program with a recognizable artifact — the code world model.

The remaining work splits into two halves. The first is empirical: close the eval gap, isolate the causal contribution of WM-training, build hybrid neural-symbolic systems whose correctness is verifier-checkable rather than test-checkable. The second is rhetorical: hold the term "world model" to a strict definition so the literature can distinguish architectural commitments from training-data choices, and resist the temptation to oversell extractability theorems and latent-imagination analogies whose premises do not transfer to code.

The opportunity is large precisely because the framework is now clear enough to identify what is missing. The work to do is the work this survey has tried to make visible.

0.22 Appendix · Glossary

- CWM — Code World Model ([2] and lineage).
- JEPA — Joint Embedding Predictive Architecture (LeCun et al.).
- RSSM — Recurrent State-Space Model (Dreamer family).
- PRM — Process Reward Model.
- SWE agent — Software-engineering agent operating on real repositories.
- Trace pretraining — Pretraining where execution traces appear in input or target.
- Execution-grounded RL — RL whose reward derives from program execution.
- Latent-imagination rollout — Forward simulation in compressed latent space rather than token space.
- TTS — Test-time scaling (sampling many candidates + verifier reranking at inference).
- GRPO — Group Relative Policy Optimization (R1-family RL algorithm).

References

[1] Wojciech Zaremba and Ilya Sutskever. Learning to execute, 2014. URL https://arxiv.org/abs/1410.4615.

[2] FAIR CodeGen team, Jade Copet, Quentin Carbonneaux, Gal Cohen, Jonas Gehring, Jacob Kahn, Jannik Kossen, Felix Kreuk, Emily McMilin, Michel Meyer, Yuxiang Wei, David Zhang, Kunhao Zheng, Jordi Armengol-Estapé, Pedram Bashiri, Maximilian Beck, Pierre Chambon, Abhishek Charnalia, Chris Cummins, Juliette Decugis, Zacharias V. Fisches, François Fleuret, Fabian Gloeckle, Alex Gu, Michael Hassid, Daniel Haziza, Badr Youbi Idrissi, Christian Keller, Rahul Kindi, Hugh Leather, Gallil Maimon, Aram Markosyan, Francisco Massa, Pierre-Emmanuel Mazaré, Vegard Mella, Naila Murray, Keyur Muzumdar, Peter O'Hearn, Matteo Pagliardini, Dmitrii Pedchenko, Tal Remez, Volker Seeker, Marco Selvi, Oren Sultan, Sida Wang, Luca Wehrstedt, Ori Yoran, Lingming Zhang, Taco Cohen, Yossi Adi, and Gabriel Synnaeve. Cwm: An open-weights llm for research on code generation with world models, 2025. URL https://arxiv.org/abs/2510.02387.

[3] Jian Wang, Xiaofei Xie, Qiang Hu, Shangqing Liu, and Yi Li. Do code semantics help? a comprehensive study on execution trace-based information for code large language models, 2025. URL https://arxiv.org/abs/2509.11686.

[4] Jonathan Richens, David Abel, Alexis Bellot, and Tom Everitt. General agents contain world models, 2025. URL https://arxiv.org/abs/2506.01622.

[5] Juyong Jiang, Fan Wang, Jiasi Shen, Sungju Kim, and Sunghun Kim. A survey on large language models for code generation, 2024. URL https://arxiv.org/abs/2406.00515.

[6] Jingtao Ding, Yunke Zhang, Yu Shang, Jie Feng, Yuheng Zhang, Zefang Zong, Yuan Yuan, Hongyuan Su, Nian Li, Jinghua Piao, Yucheng Deng, Nicholas Sukiennik, Chen Gao, Fengli Xu, and Yong Li. Understanding world or predicting future? a comprehensive survey of world models, 2024. URL https://arxiv.org/abs/2411.14499.

[7] Xinqing Li, Xin He, Le Zhang, Min Wu, Xiaoli Li, and Yun Liu. A comprehensive survey on world models for embodied ai, 2025. URL https://arxiv.org/abs/2510.16732.

[8] Chengxing Jia, Ziniu Li, Pengyuan Wang, Yi-Chen Li, Zhenyu Hou, Yuxiao Dong, and Yang Yu. Controlling large language model with latent actions, 2025. URL https://arxiv.org/abs/2503.21383.

[9] Nicola Dainese, Matteo Merler, Minttu Alakuijala, and Pekka Marttinen. Generating code world models with large language models guided by monte carlo tree search, 2024. URL https://arxiv.org/abs/2405.15383.

[10] Sergey Rodionov. Executable world models for arc-agi-3 in the era of coding agents, 2026. URL https://arxiv.org/abs/2605.05138.

[11] Scott Reed and Nando de Freitas. Neural programmer-interpreters, 2015. URL https://arxiv.org/abs/1511.06279.

[12] Ke Wang, Rishabh Singh, and Zhendong Su. Dynamic neural program embedding for program repair, 2017. URL https://arxiv.org/abs/1711.07163.

[13] Zhan Shi, Kevin Swersky, Daniel Tarlow, Parthasarathy Ranganathan, and Milad Hashemi. Learning execution through neural code fusion, 2019. URL https://arxiv.org/abs/1906.07181.

[14] David Bieber, Charles Sutton, Hugo Larochelle, and Daniel Tarlow. Learning to execute programs with instruction pointer attention graph neural networks, 2020. URL https://arxiv.org/abs/2010.12621.

[15] David Ha and Jürgen Schmidhuber. World models, 2018. URL https://arxiv.org/abs/1803.10122.

[16] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code, 2021. URL https://arxiv.org/abs/2107.03374.

[17] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models, 2021. URL https://arxiv.org/abs/2108.07732.

[18] Maxwell Nye, Anders Johan Andreassen, Guy Gur-Ari, Henryk Michalewski, Jacob Austin, David Bieber, David Dohan, Aitor Lewkowycz, Maarten Bosma, David Luan, Charles Sutton, and Augustus Odena. Show your work: Scratchpads for intermediate computation with language models, 2021. URL https://arxiv.org/abs/2112.00114.

[19] Chenxiao Liu, Shuai Lu, Weizhu Chen, Daxin Jiang, Alexey Svyatkovskiy, Shengyu Fu, Neel Sundaresan, and Nan Duan. Code execution with pre-trained language models, 2023. URL https://arxiv.org/abs/2305.05383.

[20] Yangruibo Ding, Ben Steenhoek, Kexin Pei, Gail Kaiser, Wei Le, and Baishakhi Ray. Traced: Execution-aware pre-training for source code, 2023. URL https://arxiv.org/abs/2306.07487.

[21] Alex Gu, Baptiste Rozière, Hugh Leather, Armando Solar-Lezama, Gabriel Synnaeve, and Sida I. Wang. Cruxeval: A benchmark for code reasoning, understanding and execution, 2024. URL https://arxiv.org/abs/2401.03065.

[22] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning, 2023. URL https://arxiv.org/abs/2303.11366.

[23] Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. Teaching large language models to self-debug, 2023. URL https://arxiv.org/abs/2304.05128.

[24] Ansong Ni, Srini Iyer, Dragomir Radev, Ves Stoyanov, Wen tau Yih, Sida I. Wang, and Xi Victoria Lin. Lever: Learning to verify language-to-code generation with execution, 2023. URL https://arxiv.org/abs/2302.08468.

[25] Shibo Hao, Yi Gu, Haodi Ma, Joshua Jiahua Hong, Zhen Wang, Daisy Zhe Wang, and Zhiting Hu. Reasoning with language model is planning with world model, 2023. URL https://arxiv.org/abs/2305.14992.

[26] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues?, 2023. URL https://arxiv.org/abs/2310.06770.

[27] Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. Executable code actions elicit better llm agents, 2024. URL https://arxiv.org/abs/2402.01030.

[28] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering, 2024. URL https://arxiv.org/abs/2405.15793.

[29] Ansong Ni, Miltiadis Allamanis, Arman Cohan, Yinlin Deng, Kensen Shi, Charles Sutton, and Pengcheng Yin. Next: Teaching large language models to reason about code execution, 2024. URL https://arxiv.org/abs/2404.14662.

[30] Jonas Gehring, Kunhao Zheng, Jade Copet, Vegard Mella, Quentin Carbonneaux, Taco Cohen, and Gabriel Synnaeve. Rlef: Grounding code llms in execution feedback with reinforcement learning, 2024. URL https://arxiv.org/abs/2410.02089.

[31] Yu Gu, Kai Zhang, Yuting Ning, Boyuan Zheng, Boyu Gou, Tianci Xue, Cheng Chang, Sanjari Srivastava, Yanan Xie, Peng Qi, Huan Sun, and Yu Su. Is your llm secretly a world model of the internet? model-based planning for web agents, 2024. URL https://arxiv.org/abs/2411.06559.

[32] DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, Bing Xue, Bingxuan Wang, Bochao Wu, Bei Feng, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Qu, Hui Li, Jianzhong Guo, Jiashi Li, Jiawei Wang, Jingchang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, J. L. Cai, Jiaqi Ni, Jian Liang, Jin Chen, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Liang Zhao, Litong Wang, Liyue Zhang, Lei Xu, Leyi Xia, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Meng Li, Miaojun Wang, Mingming Li, Ning Tian, Panpan Huang, Peng Zhang, Qiancheng Wang, Qinyu Chen, Qiushi Du, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, R. J. Chen, R. L. Jin, Ruyi Chen, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shengfeng Ye, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shuting Pan, S. S. Li, Shuang Zhou, Shaoqing Wu, Shengfeng Ye, Tao Yun, Tian Pei, Tianyu Sun, T. Wang, Wangding Zeng, Wanjia Zhao, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, W. L. Xiao, Wei An, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaotao Nie, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, X. Q. Li, Xiangyue Jin, Xiaojin Shen, Xiaosha Chen, Xiaowen Sun, Xiaoxiang Wang, Xinnan Song, Xinyi Zhou, Xianzu Wang, Xinxia Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Yang Zhang, Yanhong Xu, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Yu, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yuan Ou, Yuduan Wang, Yue Gong, Yuheng Zou, Yujia He, Yunfan Xiong, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Y. X. Zhu, Yanhong Xu, Yanping Huang, Yaohui Li, Yi Zheng, Yuchen Zhu, Yunxian Ma, Ying Tang, Yukun Zha, Yuting Yan, Z. Z. Ren,

Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhicheng Ma, Zhigang Yan, Zhiyu Wu, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Zizheng Pan, Zhen Huang, Zhipeng Xu, Zhongyu Zhang, and Zhen Zhang. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning, 2025. URL https://arxiv.org/abs/2501.12948.

[33] Yuxiang Wei, Olivier Duchenne, Jade Copet, Quentin Carbonneaux, Lingming Zhang, Daniel Fried, Gabriel Synnaeve, Rishabh Singh, and Sida I. Wang. Swe-rl: Advancing llm reasoning via reinforcement learning on open software evolution, 2025. URL https://arxiv.org/abs/2502.18449.

[34] Hai Huang, Yann LeCun, and Randall Balestriero. Llm-jepa: Large language models meet joint embedding predictive architectures, 2025. URL https://arxiv.org/abs/2509.14252.

[35] Babak Rahmani. Debugging code world models, 2026. URL https://arxiv.org/abs/2602.07672.

[36] Mohammad Abdollahi, Khandaker Rifah Tasnia, Soumit Kanti Saha, Jinqiu Yang, Song Wang, and Hadi Hemmati. Demystifying errors in llm reasoning traces: An empirical study of code execution simulation, 2025. URL https://arxiv.org/abs/2512.00215.

[37] Jian Yang, Wei Zhang, Jiajun Wu, Junhang Cheng, Tuney Zheng, Fanglin Xu, Weicheng Gu, Lin Jing, Yaxin Du, Joseph Li, Yizhi Li, Yan Xing, Chuan Hao, Ran Tao, Ruihao Gong, Aishan Liu, Zhoujun Li, Mingjie Tang, Chenghua Lin, Siheng Chen, Wayne Xin Zhao, Xianglong Liu, Ming Zhou, Bryan Dai, and Weifeng Lv. Incoder-32b-thinking: Industrial code world model for thinking, 2026. URL https://arxiv.org/abs/2604.03144.

[38] Gautam Singh, Arjun Guha, Bhavya Kailkhura, and Harshitha Menon. Learning reasoning world models for parallel code, 2026. URL https://arxiv.org/abs/2604.20926.

[39] Xiao Yu, Baolin Peng, Ruize Xu, Yelong Shen, Pengcheng He, Suman Nath, Nikhil Singh, Jiangfeng Gao, and Zhou Yu. Reinforcement world model learning for llm-based agents, 2026. URL https://arxiv.org/abs/2602.05842.

[40] Shansan Gong, Ruixiang Zhang, Huangjie Zheng, Jiatao Gu, Navdeep Jaitly, Lingpeng Kong, and Yizhe Zhang. Diffucoder: Understanding and improving masked diffusion models for code generation, 2025. URL https://arxiv.org/abs/2506.20639.

[41] Zhihui Xie, Jiacheng Ye, Lin Zheng, Jiahui Gao, Jingwei Dong, Zirui Wu, Xueliang Zhao, Shansan Gong, Xin Jiang, Zhenguo Li, and Lingpeng Kong. Dream-coder 7b: An open diffusion language model for code, 2025. URL https://arxiv.org/abs/2509.01142.

[42] Martin Monperrus. Bootstrapping coding agents: The specification is the program, 2026. URL https://arxiv.org/abs/2603.17399.

[43] Yangruibo Ding, Jinjun Peng, Marcus J. Min, Gail Kaiser, Junfeng Yang, and Baishakhi Ray. Semcoder: Training code language models with comprehensive semantics reasoning, 2024. URL https://arxiv.org/abs/2406.01006.

[44] Siru Ouyang, Wenhao Yu, Kaixin Ma, Zilin Xiao, Zhihan Zhang, Mengzhao Jia, Jiawei Han, Hongming Zhang, and Dong Yu. Repograph: Enhancing ai software engineering with repository-level code graph, 2024. URL https://arxiv.org/abs/2410.14684.

[45] Shangmin Guo, Omar Darwiche Domingues, Raphaël Avalos, Aaron Courville, and Florian Strub. World modelling improves language model agents, 2025. URL https://arxiv.org/abs/2506.02918.

[46] Mantas Baksys, Stefan Zetzsche, Olivier Bouissou, Remi Delmas, Soonho Kong, and Sean B. Holden. Atlas: Automated toolkit for large-scale verified code synthesis, 2025. URL https://arxiv.org/abs/2512.10173.

[47] Chuanhao Yan, Fengdi Che, Xuhan Huang, Xu Xu, Xin Li, Yizhi Li, Xingwei Qu, Jingzhe Shi, Chenghua Lin, Yaodong Yang, Binhang Yuan, Hang Zhao, Yu Qiao, Bowen Zhou, and Jie Fu. Re:form – reducing human priors in scalable formal software verification with rl in llms: A preliminary study on dafny, 2025. URL https://arxiv.org/abs/2507.16331.

[48] Amitayush Thakur, Jasper Lee, George Tsoukalas, Meghana Sistla, Matthew Zhao, Stefan Zetzsche, Greg Durrett, Yisong Yue, and Swarat Chaudhuri. Clever: A curated benchmark for formally verified code generation, 2025. URL https://arxiv.org/abs/2505.13938.

[49] Chuyue Sun, Yican Sun, Daneshvar Amrollahi, Ethan Zhang, Shuvendu Lahiri, Shan Lu, David Dill, and Clark Barrett. Veristruct: Ai-assisted automated verification of data-structure modules in verus, 2025. URL https://arxiv.org/abs/2510.25015.

[50] Haoxin Tu, Huan Zhao, Yahui Song, Mehtab Zafar, Ruijie Meng, and Abhik Roychoudhury. Agentic verification of software systems, 2025. URL https://arxiv.org/abs/2511.17330.

[51] Lingxiao Tang, He Ye, Zhaoyang Chu, Muyang Ye, Zhongxin Liu, Xiaoxue Ren, and Lingfeng Bao. Execverify: White-box rl with verifiable stepwise rewards for code execution reasoning, 2026. URL https://arxiv.org/abs/2603.11226.

[52] Shubham Gandhi, Jason Tsay, Jatin Ganhotra, Kiran Kate, and Yara Rizk. When agents go astray: Course-correcting swe agents with prms, 2025. URL https://arxiv.org/abs/2509.02360.

[53] Muhammad Khalifa, Rishabh Agarwal, Lajanugen Logeswaran, Jaekyeom Kim, Hao Peng, Moontae Lee, Honglak Lee, and Lu Wang. Process reward models that think, 2025. URL https://arxiv.org/abs/2504.16828.

[54] Junkai Chen, Zhiyuan Pan, Xing Hu, Zhenhao Li, Ge Li, and Xin Xia. Reasoning runtime behavior of a program with llm: How far are we?, 2024. URL https://arxiv.org/abs/2403.16437.

[55] Jordi Armengol-Estapé, Quentin Carbonneaux, Tianjun Zhang, Aram H. Markosyan, Volker Seeker, Chris Cummins, Melanie Kambadur, Michael F. P. O'Boyle, Sida Wang, Gabriel Synnaeve, and Hugh James Leather. What i cannot execute, i do not understand: Training and evaluating llms on program execution traces, 2025. URL https://arxiv.org/abs/2503.05703.

[56] Dongwon Jung, Wenxuan Zhou, and Muhao Chen. Code execution as grounded supervision for llm reasoning, 2025. URL https://arxiv.org/abs/2506.10343.

[57] Gallil Maimon, Ori Yoran, Felix Kreuk, Michael Hassid, Gal Cohen, Pierre Chambon, and Yossi Adi. Self-execution simulation improves coding models, 2026. URL https://arxiv.org/abs/2604.03253.

[58] Subramanyam Sahoo. The double life of code world models: Provably unmasking malicious behavior through execution traces, 2025. URL https://arxiv.org/abs/2512.13821.

[59] Maximilian Beck, Jonas Gehring, Jannik Kossen, and Gabriel Synnaeve. Towards a neural debugger for python, 2026. URL https://arxiv.org/abs/2603.09951.

[60] Mingchen Zhuge, Changsheng Zhao, Haozhe Liu, Zijian Zhou, Shuming Liu, Wenyi Wang, Ernie Chang, Gael Le Lan, Junjie Fei, Wenxuan Zhang, Yasheng Sun, Zhipeng Cai, Zechun Liu, Yunyang Xiong, Yining Yang, Yuandong Tian, Yangyang Shi, Vikas Chandra, and Jürgen Schmidhuber. Neural computers, 2026. URL https://arxiv.org/abs/2604.06425.

[61] Hyungjoo Chae, Namyoung Kim, Kai Tzu iunn Ong, Minju Gwak, Gwanwoo Song, Jihoon Kim, Sunghwan Kim, Dongha Lee, and Jinyoung Yeo. Web agents with world models: Learning and leveraging environment dynamics in web navigation, 2024. URL https://arxiv.org/abs/2410.13232.

[62] Sherry Yang. World models as an intermediary between agents and the real world, 2026. URL https://arxiv.org/abs/2602.00785.

[63] Xiao Yu, Baolin Peng, Ruize Xu, Michel Galley, Hao Cheng, Suman Nath, Jianfeng Gao, and Zhou Yu. Dyna-think: Synergizing reasoning, acting, and world model simulation in ai agents, 2025. URL https://arxiv.org/abs/2506.00320.

[64] Jiayi Pan, Xingyao Wang, Graham Neubig, Navdeep Jaitly, Heng Ji, Alane Suhr, and Yizhe Zhang. Training software engineering agents and verifiers with swe-gym, 2024. URL https://arxiv.org/abs/2412.21139.

[65] Shuang Sun, Huatong Song, Lisheng Huang, Jinhao Jiang, Ran Le, Zhihao Lv, Zongchao Chen, Yiwen Hu, Wenyang Luo, Wayne Xin Zhao, Yang Song, Hongteng Xu, Tao Zhang, and Ji-Rong Wen. Swe-world: Building software engineering agents in docker-free environments, 2026. URL https://arxiv.org/abs/2602.03419.

[66] Zhiyuan Zeng, Yichi Zhang, Yong Shan, Kai Hua, Siyuan Fang, Zhaiyu Liu, Jiaheng Liu, Haozhe Wang, Yining Zheng, Ming Ding, Ke Shen, Ge Zhang, Wenhao Huang, and Xipeng Qiu. Understanding by reconstruction: Reversing the software development process for llm pretraining, 2026. URL https://arxiv.org/abs/2603.11103.

[67] Hao Han, Jin Xie, Xuehao Ma, Weiquan Zhu, Ziyao Zhang, ZhiLiang Long, Hongkai Chen, and Qingwen Ye. Swe-trace: Optimizing long-horizon swe agents through rubric process reward models and heuristic test-time scaling, 2026. URL https://arxiv.org/abs/2604.14820.

[68] Yuxiang Wei, Zhiqing Sun, Emily McMilin, Jonas Gehring, David Zhang, Gabriel Synnaeve, Daniel Fried, Lingming Zhang, and Sida Wang. Toward training superintelligent software agents through self-play swe-rl, 2025. URL https://arxiv.org/abs/2512.18552.

[69] Zhisong Qiu, Shuofei Qiao, Kewei Xu, Yuqi Zhu, Lun Du, Ningyu Zhang, and Huajun Chen. Rewarding the scientific process: Process-level reward modeling for agentic data analysis, 2026. URL https://arxiv.org/abs/2604.24198.

[70] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models, 2023. URL https://arxiv.org/abs/2305.10601.

[71] Xidong Feng, Ziyu Wan, Muning Wen, Stephen Marcus McAleer, Ying Wen, Weinan Zhang, and Jun Wang. Alphazero-like tree-search can guide large language model decoding and training, 2023. URL https://arxiv.org/abs/2309.17179.

[72] Jing Yu Koh, Stephen McAleer, Daniel Fried, and Ruslan Salakhutdinov. Tree search for language model agents, 2024. URL https://arxiv.org/abs/2407.01476.

[73] John Schultz, Jakub Adamek, Matej Jusup, Marc Lanctot, Michael Kaisers, Sarah Perrin, Daniel Hennes, Jeremy Shar, Cannada Lewis, Anian Ruoss, Tom Zahavy, Petar Veličković, Laurel Prince, Satinder Singh, Eric Malmi, and Nenad Tomašev. Mastering board games by external and internal planning with language models, 2024. URL https://arxiv.org/abs/2412.12119.

[74] Boaz Lavon, Shahar Katz, and Lior Wolf. Execution guided line-by-line code generation, 2025. URL https://arxiv.org/abs/2506.10948.

[75] Shuocheng Li, Yihao Liu, Silin Du, Wenxuan Zeng, Zhe Xu, Mengyu Zhou, Yeye He, Haoyu Dong, Shi Han, and Dongmei Zhang. Jupiter: Enhancing llm data analysis capabilities via notebook and inference-time value-guided search, 2025. URL https://arxiv.org/abs/2509.09245.

[76] Anthony Z. Liu, Xinhe Wang, Jacob Sansom, Yao Fu, Jongwook Choi, Sungryull Sohn, Jaekyeom Kim, and Honglak Lee. Interactive and expressive code-augmented planning with large language models, 2024. URL https://arxiv.org/abs/2411.13826.

[77] Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, and Nicolas Ballas. Self-supervised learning from images with a joint-embedding predictive architecture, 2023. URL https://arxiv.org/abs/2301.08243.

[78] Danijar Hafner, Timothy Lillicrap, Jimmy Ba, and Mohammad Norouzi. Dream to control: Learning behaviors by latent imagination, 2019. URL https://arxiv.org/abs/1912.01603.

[79] Danijar Hafner, Timothy Lillicrap, Mohammad Norouzi, and Jimmy Ba. Mastering atari with discrete world models, 2020. URL https://arxiv.org/abs/2010.02193.

[80] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse domains through world models, 2023. URL https://arxiv.org/abs/2301.04104.

[81] Yuan Pu, Yazhe Niu, Zhenjie Yang, Jiyuan Ren, Hongsheng Li, and Yu Liu. Unizero: Generalized and efficient planning with scalable latent world models, 2024. URL https://arxiv.org/abs/2406.10667.

[82] Jake Bruce, Michael Dennis, Ashley Edwards, Jack Parker-Holder, Yuge Shi, Edward Hughes, Matthew Lai, Aditi Mavalankar, Richie Steigerwald, Chris Apps, Yusuf Aytar, Sarah Bechtle, Feryal Behbahani, Stephanie Chan, Nicolas Heess, Lucy Gonzalez, Simon Osindero, Sherjil Ozair, Scott Reed, Jingwei Zhang, Konrad Zolna, Jeff Clune, Nando de Freitas, Satinder Singh, and Tim Rocktäschel. Genie: Generative interactive environments, 2024. URL https://arxiv.org/abs/2402.15391.

[83] Tristan Kenneweg, Philip Kenneweg, and Barbara Hammer. Jepa for rl: Investigating joint-embedding predictive architectures for reinforcement learning, 2025. URL https://arxiv.org/abs/2504.16591.

[84] Hanzhuo Tan, Weihao Li, Xiaolong Tian, Siyi Wang, Jiaming Liu, Jing Li, and Yuqun Zhang. Sk2decompile: Llm-based two-phase binary decompilation from skeleton to skin, 2025. URL https://arxiv.org/abs/2509.22114.

[85] Yongpan Wang, Xin Xu, Xiaojie Zhu, Xiaodong Gu, and Beijun Shen. Salt4decompile: Inferring source-level abstract logic tree for llm-based binary decompilation, 2025. URL https://arxiv.org/abs/2509.14646.

[86] Anjiang Wei, Jiannan Cao, Ran Li, Hongyu Chen, Yuhui Zhang, Ziheng Wang, Yuan Liu, Thiago S. F. X. Teixeira, Diyi Yang, Ke Wang, and Alex Aiken. Equibench: Benchmarking large language models' reasoning about program semantics via equivalence checking, 2025. URL https://arxiv.org/abs/2502.12466.

[87] Fu Teng, Miao Pan, Xuhong Zhang, Zhezhi He, Yiyao Yang, Xinyi Chai, Mengnan Qi, Liqiang Lu, and Jianwei Yin. Verirl: Boosting the llm-based verilog code generation via reinforcement learning, 2025. URL https://arxiv.org/abs/2508.18462.

[88] Zhirong Chen, Kaiyan Chang, Zhuolin Li, Cangyuan Li, Xinyang He, Chujie Chen, Mengdi Wang, Haobo Xu, Yinhe Han, Huawei Li, and Ying Wang. Chipseek: Optimizing verilog generation via eda-integrated reinforcement learning, 2025. URL https://arxiv.org/abs/2507.04736.

[89] Anjiang Wei, Huanmi Tan, Tarun Suresh, Daniel Mendoza, Thiago S. F. X. Teixeira, Ke Wang, Caroline Trippel, and Alex Aiken. Vericoder: Enhancing llm-based rtl code generation through functional correctness validation, 2025. URL https://arxiv.org/abs/2504.15659.

[90] Julien Pourcel, Cédric Colas, and Pierre-Yves Oudeyer. Self-improving language models for evolutionary program synthesis: A case study on arc-agi, 2025. URL https://arxiv.org/abs/2507.14172.

[91] Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. Darwin godel machine: Open-ended evolution of self-improving agents, 2025. URL https://arxiv.org/abs/2505.22954.

[92] Wenyi Wang, Piotr Piękos, Li Nanbo, Firas Laakom, Yimeng Chen, Mateusz Ostaszewski, Mingchen Zhuge, and Jürgen Schmidhuber. Huxley-gödel machine: Human-level coding agent development by an approximation of the optimal self-improving machine, 2025. URL https://arxiv.org/abs/2510.21614.

[93] Yuxiang Zhang, Shangxi Wu, Yuqi Yang, Jiangming Shu, Jinlin Xiao, Chao Kong, and Jitao Sang. o1-coder: an o1 replication for coding, 2024. URL https://arxiv.org/abs/2412.00154.

[94] Yongchao Chen, Yueying Liu, Junwei Zhou, Yilun Hao, Jingquan Wang, Yang Zhang, Na Li, and Chuchu Fan. R1-code-interpreter: Llms reason with code via supervised and multi-stage reinforcement learning, 2025. URL https://arxiv.org/abs/2505.21668.

[95] Mehrzad Samadi, Aleksander Ficek, Sean Narenthiran, Siddhartha Jain, Wasi Uddin Ahmad, Somshubra Majumdar, Vahid Noroozi, and Boris Ginsburg. Scaling test-time compute to achieve ioi gold medal with open-weight models, 2025. URL https://arxiv.org/abs/2510.14232.

[96] Mathis Pink, Qinyuan Wu, Vy Ai Vo, Javier Turek, Jianing Mu, Alexander Huth, and Mariya Toneva. Position: Episodic memory is the missing piece for long-term llm agents, 2025. URL https://arxiv.org/abs/2502.06975.

[97] Yuxiang Zhang, Jiangming Shu, Ye Ma, Xueyuan Lin, Shangxi Wu, and Jitao Sang. Memory as action: Autonomous context curation for long-horizon agentic tasks, 2025. URL https://arxiv.org/abs/2510.12635.

[98] Antonio Valerio Miceli Barone and Poon Tsz Nok. Improving llm code reasoning via semantic equivalence self-play with formal verification, 2026. URL https://arxiv.org/abs/2604.17010.

[99] Jacqueline L. Mitchell, Brian Hyeongseok Kim, Chenyu Zhou, and Chao Wang. Understanding formal reasoning failures in llms as abstract interpreters, 2025. URL https://arxiv.org/abs/2503.12686.

[100] Yihe Li, Ruijie Meng, and Gregory J. Duck. Large language model powered symbolic execution, 2025. URL https://arxiv.org/abs/2505.13452.

[101] Fanpeng Yang, Xu Ma, Shuling Wang, Xiong Xu, Qinxiang Cao, Naijun Zhan, Xiaofeng Li, and Bin Gu. Integrating symbolic execution with llms for automated generation of program specifications, 2025. URL https://arxiv.org/abs/2506.09550.

[102] Wenhan Wang, Kaibo Liu, An Ran Chen, Ge Li, Zhi Jin, Gang Huang, and Lei Ma. Python symbolic execution with llm-powered code generation, 2024. URL https://arxiv.org/abs/2409.09271.

[103] Varun Bharti, Shashwat Jha, Dhruv Kumar, and Pankaj Jalote. Loop invariant generation: A hybrid framework of reasoning optimised llms and smt solvers, 2025. URL https://arxiv.org/abs/2508.00419.

[104] Kriz Tahimic and Charibeth Cheng. Mechanistic interpretability of code correctness in llms via sparse autoencoders, 2025. URL https://arxiv.org/abs/2510.02917.

[105] Francisco Ribeiro, Claudio Spiess, Prem Devanbu, and Sarah Nadi. On llms' internal representation of code correctness, 2025. URL https://arxiv.org/abs/2512.07404.

[106] Yunkun Wang, Yue Zhang, Guochang Li, Chen Zhi, Binhua Li, Fei Huang, Yongbin Li, and Shuiguang Deng. Inspectcoder: Dynamic analysis-enabled self repair through interactive llm-debugger collaboration, 2025. URL https://arxiv.org/abs/2510.18327.

[107] Zhengyao Liu, Yunlong Ma, Jingxuan Xu, Junchen Ai, Xiang Gao, Hailong Sun, and Abhik Roychoudhury. Agent that debugs: Dynamic state-guided vulnerability repair, 2025. URL https://arxiv.org/abs/2504.07634.

[108] Shubham Ugare and Satish Chandra. Agentic code reasoning, 2026. URL https://arxiv.org/abs/2603.01896.

[109] Shenao Yan, Shen Wang, Yue Duan, Hanbin Hong, Kiho Lee, Doowon Kim, and Yuan Hong. An llm-assisted easy-to-trigger backdoor attack on code completion models: Injecting disguised vulnerabilities against strong detection, 2024. URL https://arxiv.org/abs/2406.06822.

[110] George Edwards and Mahdi Eslamimehr. Synergistic directed execution and llm-driven analysis for zero-day ai-generated malware detection, 2026. URL https://arxiv.org/abs/2603.09044.

[111] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code, 2024. URL https://arxiv.org/abs/2403.07974.

[112] Ruiyang Xu, Jialun Cao, Yaojie Lu, Ming Wen, Hongyu Lin, Xianpei Han, Ben He, Shing-Chi Cheung, and Le Sun. Cruxeval-x: A benchmark for multilingual code reasoning, understanding and execution, 2024. URL https://arxiv.org/abs/2408.13001.

[113] Yikun Li, Jinfeng Jiang, Ting Zhang, Chengran Yang, Chenxing Zhong, Yin Yide, Leow Wen Bin, Eng Lieh Ouh, Lwin Khin Shar, and David Lo. An execution-verified multi-language benchmark for code semantic reasoning, 2026. URL https://arxiv.org/abs/2605.11006.

[114] Aditya Thimmaiah, Jiyang Zhang, Jayanth Srinivasa, Junyi Jessy Li, and Milos Gligoric. Plsemanticsbench: Large language models as programming language interpreters, 2025. URL https://arxiv.org/abs/2510.03415.

[115] Anjiang Wei, Tarun Suresh, Jiannan Cao, Naveen Kannan, Yuheng Wu, Kai Yan, Thiago S. F. X. Teixeira, Ke Wang, and Alex Aiken. Codearc: Benchmarking reasoning capabilities of llm agents for inductive program synthesis, 2025. URL https://arxiv.org/abs/2503.23145.

[116] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. Webarena: A realistic web environment for building autonomous agents, 2023. URL https://arxiv.org/abs/2307.13854.

[117] Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Samuel Stevens, Boshi Wang, Huan Sun, and Yu Su. Mind2web: Towards a generalist agent for the web, 2023. URL https://arxiv.org/abs/2306.06070.

[118] Yaolun Zhang, Yinxu Pan, Yudong Wang, and Jie Cai. Pybench: Evaluating llm agent on various real-world coding tasks, 2024. URL https://arxiv.org/abs/2407.16732.

[119] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. Autocoderover: Autonomous program improvement, 2024. URL https://arxiv.org/abs/2404.05427.

[120] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Agentless: Demystifying llm-based software engineering agents, 2024. URL https://arxiv.org/abs/2407.01489.

[121] Jeff Da, Clinton Wang, Xiang Deng, Yuntao Ma, Nikhil Barhate, and Sean Hendryx. Agent-rlvr: Training software engineering agents via guidance and environment rewards, 2025. URL https://arxiv.org/abs/2506.11425.

[122] Alexander Golubev, Maria Trofimova, Sergei Polezhaev, Ibragim Badertdinov, Maksim Nekrashevich, Anton Shevtsov, Simon Karasik, Sergey Abramov, Andrei Andriushchenko, Filipp Fisin, Sergei Skvortsov, and Boris Yangel. Training long-context, multi-turn software engineering agents with reinforcement learning, 2025. URL https://arxiv.org/abs/2508.03501.

[123] Maxime Robeyns, Martin Szummer, and Laurence Aitchison. A self-improving coding agent, 2025. URL https://arxiv.org/abs/2504.15228.

[124] Ruiyi Zhang, Peijia Qin, Qi Cao, Eric Xue, and Pengtao Xie. Funprm: Function-as-step process reward model with meta reward correction for code generation, 2026. URL https://arxiv.org/abs/2601.22249.

[125] Theo X. Olausson, Jeevana Priya Inala, Chenglong Wang, Jianfeng Gao, and Armando Solar-Lezama. Is self-repair a silver bullet for code generation?, 2023. URL https://arxiv.org/abs/2306.09896.